C# и .NET — методичка и практикум

От первой программы к разработке и сопровождению приложений

C# 14 · .NET 10 LTS
Редакция 2026-10-22

Последовательное изучение языка C# и платформы .NET. Теория, самостоятельные задания и сквозной проект. Первый выпуск: главы 01–10, лабораторная L01 и программа из 60 глав. Главы 11–60 пока представлены планом. Все написанные учебные страницы включены в эту книгу.

Оглавление

Как проходить практикум C# и .NET 3

01. C# и .NET, рабочая среда 5

02. Синтаксис и диагностика 8

03. Типы, числа и операции 11

04. Ветвления и циклы 14

05. Методы и декомпозиция 17

06. Строки, null и культура 20

07. Массивы и базовые коллекции 23

08. Классы и инварианты 25

09. Ошибки, ресурсы и первые файлы 28

10. Практикум ExpenseLedger CLI 31

L01. Консольный журнал расходов 33

Подсказки и разбор контрольных вопросов 36

Карта покрытия языка C# 38

Источники и версии 40

Программа практикума C# и .NET 42

Как проходить практикум C# и .NET

Этот курс учит писать, проверять и сопровождать программы на C#. Начинать можно без другого языка программирования; достаточно уметь работать с файлами и терминалом. Первая цель — самостоятельно сделать консольный отчёт по расходам. Затем программа развивается в библиотеку, импортёр, сервис с базой данных и клиент выбранной специализации.

Что уже можно изучать

Полностью подготовлен первый блок: главы 01–10, задания, подсказки, запускаемые примеры и лабораторная L01. Продолжение подробно расписано в программе; будущие главы отмечены как план. Это позволяет проходить материал последовательно и видеть, какие знания ещё предстоит получить.

Один учебный цикл

1. Прочитайте цель главы и попробуйте ответить на контрольные вопросы.
2. Перепечатайте небольшой пример и до запуска предскажите результат.
3. Измените одно условие: пустой ввод, границу диапазона, тип или порядок действий.
4. Решите A и B самостоятельно. В C переносите приём на изменившееся требование.
5. Запишите вход, ожидаемый и фактический результат. Объясните расхождение, если оно есть.
6. Сверьтесь с подсказками и повторите задачу без готового решения.

Удобный ритм занятия — 15–25 минут чтения, 40–60 минут кода, 10 минут разбора. Это ориентир для одного подхода, а не обещание освоить главу за час. Если не удаётся объяснить строку программы, уменьшите пример до нескольких инструкций.

Уровни практики

Уровень	Что требуется	Когда задача решена
A	Применить один приём по образцу	Результат совпал, каждая строка объяснима
B	Самому выбрать и связать несколько приёмов	Пройдены обычные и граничные случаи
C	Изменить требования или диагностировать дефект	Старое поведение сохранено, новое проверено
L	Собрать законченный этап проекта	Выполнен весь контракт, другой человек может запустить программу

Для B и C проверяйте хотя бы обычный случай, нижнюю и верхнюю границы, ошибочный ввод. Для коллекций добавляйте пустой набор и дубликат; для файлов — отсутствие и повреждение; для async в последующих блоках — отмену и частичный сбой.

Рабочие файлы

Главы находятся в `docs`, законченные примеры — в `examples`, условия лабораторных — в `labs`. Свои решения создавайте в `work/имя-задачи`; эта папка не включается в Git. Исходный `ConsoleApp1` — отдельный учебный проект пользователя. Примеры курса собираются своей solution.

Команды в тексте выполняются из корня репозитория, если явно не сказано иначе. В Windows подходят PowerShell и терминал IDE. Пути в командах используют `/`, который понимает .NET. Перед изменением примера полезно сделать копию в `work`.

Быстрый маршрут при опыте другого языка

Прочитайте 01, выполните B и C из 03, 05–09 и сдайте L01. Если все критерии выполнены, можно переходить к системе типов. Особое внимание уделите `decimal`, nullable-аннотациям, копированию ссылок, `IDisposable` и культурно зависимому разбору строк: привычки другого языка здесь часто приводят к ошибкам.

Как пользоваться книгами

Книги из `Books` — дополнительное углубление. Для первых глав они не обязательны. Книга Рихтера полезна при изучении CLR, типов и ресурсов. Книги Freeman и Yan нужны для веб-раздела. Версии ASP.NET Core 6/8 в этих изданиях отличаются от платформы курса, поэтому команды и API сверяются с актуальной документацией. Названия файлов и назначение приведены в списке источников.

Проверка первого результата

После главы 10 вы должны без образца разобрать строку расхода, выбрать тип суммы, объяснить поведение общей ссылки на список, закрыть файл при ошибке и вернуть понятный код завершения. Если какой-то пункт вызывает затруднение, вернитесь к соответствующей главе. Готовность определяется этими действиями, а не количеством прочитанных страниц.

Начать главу 01

01. C# и .NET, рабочая среда

Результат: вы создаёте проект, собираете и запускаете его из терминала и понимаете, какой инструмент участвует на каждом шаге. Предыдущие знания: маршрут.

Язык, платформа и библиотека

C# задаёт правила исходного кода: какие значения существуют, как вызвать метод и как описать тип. Компилятор Roslyn проверяет код и обычно создаёт сборку с промежуточным кодом IL и метаданными. Среда CLR загружает сборку, управляет выполнением, исключениями и памятью. JIT обычно переводит IL нужных методов в машинный код во время работы. Позже мы разберём предварительную компиляцию, в том числе Native AOT: схема исполнения не всегда ограничивается JIT.

.NET включает среду выполнения и библиотеки. Например, `if` — конструкция языка, а `Console.WriteLine` — метод библиотеки. ASP.NET Core, EF Core и WPF решают прикладные задачи; они не являются частью синтаксиса C#.

```
Program.cs + .csproj
        ↓ dotnet build / Roslyn
сборка .dll с IL и метаданными
        ↓ dotnet run / CLR и JIT
выполняющаяся программа
```

SDK содержит средства разработки и сборки. Runtime нужен для выполнения уже собранного framework-dependent приложения. Наличие runtime не гарантирует, что можно собрать проект. Современный .NET работает на нескольких ОС; .NET Framework — отдельная Windows-платформа, важная при сопровождении старых систем.

Среда курса

Используем .NET 10 LTS и C# 14. Соответствие подтверждено в описании C# 14; срок поддержки .NET 10 до 14 ноября 2028 указан в политике .NET. Дата выбора среды: 22 октября 2026. Preview-функции для упражнений не нужны.

Проверьте установку:

```
dotnet --info
dotnet --list-sdks
```

Нужен SDK `10.0.x`. Если его нет, установите .NET 10 SDK. Редактор выбирайте по удобству: Visual Studio с поддержкой .NET 10, Rider или VS Code с C#-инструментами. Обязательный минимум курса воспроизводится CLI и не зависит от конкретной IDE.

Первая программа

Создайте новый проект в папке собственных решений:

```
dotnet new console -n HelloCourse -o work/HelloCourse --framework net10.0
dotnet build work/HelloCourse
dotnet run --project work/HelloCourse
```

В `Program.cs` замените содержимое:

```
Console.WriteLine("Начинаю изучать C#");
Console.WriteLine(7 + 5);
```

Получатся две строки: сообщение и 12. Одна инструкция заканчивается ;. Вызов метода указывает действие, аргумент в скобках — данные для него. Кавычки обозначают строковый литерал; без кавычек 7 + 5 вычисляется как сумма.

Проект содержит настройки, например:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
</Project>
```

TargetFramework (TFM) выбирает платформу и доступные API. Nullable включает анализ возможного null. ImplicitUsings подключает распространённые пространства имён: поэтому в первой строке не пришлось писать System.Console. LangVersion отдельно выбирает версию синтаксиса. Менять её на preview или latest для выполнения базовых заданий не требуется.

Что делают команды

Команда	Назначение
dotnet restore	Разрешить зависимости проекта и подготовить assets
dotnet build	Проверить и скомпилировать
dotnet run	Собрать при необходимости и запустить
dotnet test	Выполнить тестовые проекты через настроенный test runner
dotnet publish	Подготовить файлы для поставки приложения

Обычно build и run вызывают restore автоматически. Пакет — внешняя зависимость, namespace — группа имён, а assembly — единица сборки и загрузки. Это разные понятия. О поведении создания проектов см. dotnet new.

Solution объединяет несколько проектов для работы в IDE и CLI. Для курса используется examples/CSharpCourse.slnx. В bin находятся результаты сборки, в obj — промежуточные данные. Эти каталоги не редактируют как исходники.

Запуск готового примера

```
dotnet run --project examples/Foundations -- 01
```

-- отделяет параметры приложения от параметров dotnet run. Проверьте, что получены Hello, C#! и 12. Затем запустите пример из IDE с breakpoint на выводе суммы и выполните одну инструкцию за раз.

Практика

01-A. Создайте HelloCourse, выведите своё имя и результат 17 * 3. Критерий: две отдельные строки, вторая — 51; сборка без ошибок.

01-B. Создайте вторую программу Distance: скорость 72 км/ч, время 3 часа. Критерий: строка Distance: 216 km. Отдельно запустите build, затем run с --no-build. Объясните, что произойдёт, если изменить исходник и сразу повторить --no-build.

01-C. Подготовьте публикацию `Distance` командой `dotnet publish work/Distance -c Release -o work/publish-distance`. Запустите `dotnet work/publish-distance/Distance.dll`. Критерий: тот же результат, вы можете указать папку проекта и папку поставки. Это framework-dependent публикация: на другой машине потребуется подходящий runtime.

Проверка понимания

Почему наличие C#-файла недостаточно для запуска? Чем SDK отличается от runtime? Где определяется TFM? Почему установка пакета не равна `using`? Вернитесь к схеме и объясните её без чтения текста.

Подсказки · Далее 02

02. Синтаксис и диагностика

Результат: вы читаете короткую программу, находите область видимости переменной и различаете ошибку компиляции, исключение и неверный результат. Нужна глава 01.

Из чего состоит код

Идентификатор — имя: `total`, `Calculate`, `Expense`. Регистр существенен: `total` и `Total` различаются. Ключевые слова, например `int` и `return`, имеют заданный смысл. Встречающееся в чужом коде `@class` — идентификатор с экранированным ключевым словом. Для собственных имён выбирайте понятные слова: `expenseCount` точнее, чем `x1`.

Литерал задаёт значение прямо в коде: `42`, `"food"`, `true`. Выражение вычисляет значение: `price * count`. Инструкция выполняет действие: `Console.WriteLine(total);`. Блок в фигурных скобках объединяет инструкции.

```
int price = 25;
int count = 4;
int total = price * count;
Console.WriteLine(total); // 100
```

= присваивает значение, а == сравнивает. Комментарии `//` идут до конца строки; `/* ... */` могут занимать несколько строк. Поясняйте в комментариях причину решения: пересказ `count++` словами «увеличить count» редко помогает.

Переменные и область видимости

Объявление связывает имя с типом. До чтения локальной переменной компилятор должен доказать, что ей присвоено значение. Поля и элементы массивов имеют значения по умолчанию, но это не разрешает читать неинициализированную локальную переменную.

```
int total = 10;
{
    int bonus = 2;
    total += bonus;
}
Console.WriteLine(total); // 12
// Console.WriteLine(bonus); // ошибка: bonus вне области видимости
```

Область видимости определяет, где доступно имя; время жизни — как долго существует значение или объект. Эти понятия связаны, но не совпадают. В дальнейшем замыкание может удерживать объект после выхода из блока.

Точка входа

В новых консольных шаблонах используется top-level код. Компилятор создаёт точку входа. В проекте может быть только один файл с такими инструкциями; они идут перед объявлениями типов в этом файле. Вторую программу нельзя просто дописать в другой файл того же проекта.

Традиционная форма того же начала программы:

```
namespace Course;

internal class Program
{
    private static void Main(string[] args)
    {
        Console.WriteLine("Hello");
    }
}
```

Это альтернативный полный `Program.cs`, а не дополнение к top-level инструкциям. `namespace` группирует имена, `class` объявляет тип, `Main` — метод точки входа. `args` содержит аргументы командной строки. Значение `static` разберём в главах 05 и 08. `using System;` позволяет писать `Console` вместо `System.Console`.

Три вида дефектов

Вид	Пример	Как искать
Ошибка компиляции	Неизвестное имя, отсутствует `;`	Сначала первая содержательная диагностика build
Ошибка выполнения	Обращение за границы массива	Тип исключения, сообщение и стек вызовов
Логическая ошибка	Вместо скидки добавили наценку	Проверка на заранее рассчитанных данных

Например, `int total = "42";` не компилируется: строку нельзя неявно присвоить целому. Вычисление `10 / divisor` при `int divisor = 0` выбросит исключение во время выполнения. А `double average = 5 / 2;` выполнится и даст 2: деление сначала было целочисленным. Исправление — `5.0 / 2`, а не смена текста вывода.

Работа отладчиком

Поставьте breakpoint перед вычислением `total`. Запустите отладку. В Locals или Watch посмотрите `price` и `count`, затем выполните Step Over. Проверьте новое значение `total`. Step Into входит в вызываемый метод, Step Out завершает текущий. Call Stack показывает путь вызовов.

Запись ошибки должна содержать: входные данные, ожидаемый результат, фактический результат, место расхождения и причину. «Не работает» не даёт другому человеку воспроизводимого сценария. Если несколько ошибок возникают после одной пропущенной скобки, сначала исправьте её и пересоберите.

Практика

02-A. Запустите `dotnet run --project examples/Foundations -- 02`. До запуска вычислите значения из исходника. Критерий: результат `total=100`.

02-B. В своей копии по очереди внесите три дефекта: переименуйте `price` только в объявлении; поделите на переменную со значением ноль; замените `*` на `+`. Критерий: для каждого дефекта записаны категория, средство обнаружения и исправление. Не оставляйте намеренно сломанный код в проекте с другими упражнениями.

02-C. Поставьте условный breakpoint перед выводом: `total > 90`. Для `price=25, count=4` он должен сработать, для `price=10, count=4` — нет. Критерий: объясните состояние переменных в каждой точке, не добавляя диагностический вывод.

Проверка понимания

Можно ли прочитать локальную переменную без присваивания? Почему переменная внутри блока недоступна снаружи? Почему программа с логической ошибкой может успешно собраться? Чем top-level код отличается от `Main` для читателя?

Официальное описание: структура программы.

Подсказки · Далее 03

03. Типы, числа и операции

Результат: вы выбираете тип данных, предсказываете арифметику и проверяете переполнение. Нужна глава 02.

Тип задаёт допустимые действия

`int` хранит целое, `bool` — `true` или `false`, `string` — текст. Тип определяет диапазон, операции и правила преобразования. `int` — псевдоним `System.Int32`, а не другой тип. `var count = 3;` тоже объявляет `int`: компилятор вывел тип из выражения. Позже присвоить этой переменной строку нельзя. `var` не равен `dynamic`.

Тип	Свойства	Типичное применение
`sbyte/byte`	8 бит, со знаком/без знака	Компактные значения, байты
`short/ushort`	16 бит	Форматы и совместимость
`int/uint`	32 бита	Счётчики, целые значения
`long/ulong`	64 бита	Большие счётчики
`nint/nuint`	Размер платформенного указателя	Interop
`float/double`	Двоичная плавающая точка, около 6–9/15–17 значащих цифр	Измерения и расчёты
`decimal`	Десятичное представление, 28–29 значащих цифр	Суммы с десятичными долями
`char`	16-битная кодовая единица UTF-16	Работа с текстовым представлением
`bool`	Два логических значения	Условия

Для `int` диапазон от −2 147 483 648 до 2 147 483 647. Значения `MinValue` и `MaxValue` лучше использовать в проверках, чем перепечатывать границы. Выбор беззнакового типа сам по себе не валидирует бизнес-правило «количество положительное»: ноль по-прежнему допустим.

Литералы, константы и преобразования

```
int count = 1_000;
long population = 8_000_000_000L;
double ratio = 0.5;
float scale = 0.5f;
decimal price = 19.90m;
const int MaxItems = 100;
int empty = default; // 0
Console.WriteLine(price * 3); // числовое значение 59.70
```

Суффикс `m` создаёт `decimal`. Без него `19.90` — `double`. `const` требует вычислимое при компиляции значение. Понятие `readonly` относится к полям и будет рассмотрено отдельно.

Неявное преобразование `int → long` сохраняет значение. Обратное обычно требует явного cast: `int small = checked((int)large);`. Явность не гарантирует отсутствие потери данных. Даже некоторые неявные преобразования к floating-point теряют точность. Подробности: числовые преобразования.

`(int)3.9` даёт `3`, а `(int)(-3.9)` — `-3`: дробь отбрасывается к нулю. Это не правило округления денежных значений. Для округления используйте `Math.Round` и явно выберите правило, например `MidpointRounding.ToEven`.

Деление, округление и переполнение

```
Console.WriteLine(5 / 2);        // 2
Console.WriteLine(5 / 2.0);      // 2.5
Console.WriteLine(5 % 2);        // 1
Console.WriteLine(0.1 + 0.2 == 0.3);    // False
Console.WriteLine(0.1m + 0.2m == 0.3m); // True

int value = int.MaxValue;
int wrapped = unchecked(value + 1); // int.MinValue
// int failed = checked(value + 1); // OverflowException
```

Сначала выполняется операция с типами её операндов, затем присваивание результату. Поэтому `long total = count * price;` не спасает от переполнения умножения двух `int`. Исправление — `checked((long)count * price)`.

`decimal` удобен для десятичных величин, но тоже имеет конечную точность: `1m / 3m` не представляется точно. Валюта, число десятичных знаков и момент округления остаются правилами приложения. Операции `decimal` при выходе за диапазон выбрасывают исключение и внутри `unchecked`.

`checked` действует на находящиеся в нём операции, а не автоматически на тело вызванного метода. Переполнение целочисленного выражения, вычислимого при компиляции, обычно выявляется уже компилятором.

Логика, приоритет и биты

`&&` и `||` применяют короткое замыкание: правый операнд вычисляется только при необходимости. Выражение `denominator != 0 && numerator / denominator > 2` не делит на ноль. `&` и `|` над `bool` вычисляют обе стороны; над целыми они выполняют побитовые операции. `^` — XOR, а не степень; для степени служит `Math.Pow`.

`1 << 3` даёт 8. В `flags & 4` проверяется конкретный бит; это не логическое «и» двух условий. У `>>` и `>>>` различается заполнение старших битов для отрицательных чисел. Подробные приёмы понадобятся в главе о enum и битовых флагах.

В `2 + 3 * 4` умножение выполняется раньше сложения. Скобки `(2 + 3) * 4` меняют результат. Не смешивайте изменение переменной через `++` и её несколько чтений в одном сложном выражении: разнесённые инструкции проще проверить.

Разбор расчёта

Стоимость трёх товаров по `19.90m` равна `59.70m`. При скидке 10%:

```
decimal subtotal = 19.90m * 3;
decimal total = Math.Round(subtotal * 0.90m, 2, MidpointRounding.ToEven);
Console.WriteLine(total); // числовое значение 53.73
```

Текстовое представление точки или запятой зависит от культуры; глава 06 закрепит формат вывода. Запуск примера: `dotnet run --project examples/Foundations -- 03`.

Практика

03-A. До запуска предскажите `7 / 3`, `7 / 3m`, `-7 % 3`, `(int)2.9`. Критерий: объяснены тип и значение каждого результата, а не только скопирован вывод.

03-B. Рассчитайте заказ: цена `125.50m`, количество 4, скидка `15m` процентов. Округление итоговой суммы — два знака, ToEven. Критерий: `426.70`. Проверьте скидки 0 и 100, количество 1. Проверку неверных входов добавите после главы 04.

03-C. Для `int count = 50_000; int price = 50_000;` получите точное значение `2_500_000_000` типа `long`. Покажите вариант, где `checked` обнаруживает ошибку, и вариант, где преобразование до умножения её предотвращает.

Проверка понимания

Почему `var` сохраняет статическую типизацию? Почему `double` не подходит для точного равенства десятичных сумм? Чем округление отличается от cast? Почему изменение типа результата после операции может быть слишком поздним?

Подсказки · Далее 04

04. Ветвления и циклы

Результат: программа принимает решения, повторяет действия и заканчивает обработку при конце ввода. Нужна глава 03.

Условие и выбор

Условие `if` имеет тип `bool`. Число `1` не преобразуется в истину автоматически.

```
int age = 20;
if (age < 0)
{
    Console.WriteLine("Некорректный возраст");
}
else if (age < 18)
{
    Console.WriteLine("Несовершеннолетний");
}
else
{
    Console.WriteLine("Совершеннолетний");
}
```

Порядок веток существенен: выполняется первая подходящая. Для интервала от 1 до 100 включительно нужно `value >= 1 && value <= 100`; запись `1 <= value <= 100` неверна.

Оператор `switch` удобен для конечного набора команд. Выражение `switch` возвращает значение:

```
string command = "sum";
string description = command switch
{
    "sum" => "Сумма",
    "count" => "Количество",
    _ => "Неизвестная команда"
};
Console.WriteLine(description);
```

В `switch statement` ветка с инструкциями обычно заканчивается `break`, `return` или `throw`. Неявного проваливания в следующую непустую ветку нет. `_ =>` в switch expression — запасной случай. Развитые шаблоны будут в главе 17.

Цикл и его границы

`for` подходит, когда известен счётчик:

```
int total = 0;
for (int i = 1; i <= 5; i++)
{
    total += i;
}
Console.WriteLine(total); // 15
```

Перед каждой итерацией проверяется условие. После тела выполняется `i++`. Инвариант этого цикла: перед итерацией с номером `i` переменная total хранит сумму от 1 до `i - 1`. Это утверждение позволяет объяснить результат и найти ошибку границы.

`while` проверяет условие перед телом, `do ... while` — после, поэтому do выполняется хотя бы раз. `foreach` последовательно получает элементы коллекции: например, `foreach (char letter in "cat") Console.WriteLine(letter);`.

`break` завершает ближайший цикл, `continue` переходит к следующей итерации, `return` выходит из метода целиком. В цикле while перед continue нужно обеспечить продвижение: иначе значение, вызвавшее continue, может обрабатываться бесконечно. В чужом коде встречается `goto`; он переносит управление к разрешённой метке. Для обычного ввода понятнее цикл и return; специальные ограничения goto разбираются по справочнику.

Конец потока и пользовательская команда

`Console.ReadLine()` возвращает строку либо `null`, если поток закончился. Это не то же самое, что пустая строка после Enter. Обозначение `string?` допускает отсутствие строки; подробно оно рассматривается в главе 06.

Полный пример для собственного `Program.cs`:

```
long total = 0;
while (true)
{
    string? line = Console.ReadLine();
    if (line is null || line == "stop")
    {
        break;
    }

    if (!int.TryParse(line, out int value))
    {
        Console.WriteLine("error");
        continue;
    }

    total = checked(total + value);
}
Console.WriteLine(total);
```

`TryParse` возвращает успех и через `out` записывает число. Не пытайтесь использовать полученное число при `false`. Здесь текст `abc` — ожидаемый неверный ввод, поэтому он обрабатывается обычной веткой. Если гигантский поток переполнит `long`, checked выявит ошибку; стратегию обработки исключений изучим в главе 09.

Для входа `10`, `abc`, `-3`, `stop` вывод — `error`, затем `7`. Вводить строки нужно по отдельности. EOF можно проверить передачей файла стандартному вводу; при интерактивном вводе Windows обычно используют Ctrl+Z и Enter.

Проверка границ

Для задачи «сумма от 1 до n» проверьте `n=0`, `1`, `5`, `-1`. Заранее определите контракт: здесь ноль даёт ноль, отрицательное n — ошибку. Тест `n=5` один не выявит все ошибки. Условия `i < n` и `i <= n` отличаются ровно одним элементом, который может потеряться незаметно.

Практика

04-A. Напишите сумму от 1 до n через for. Вход задайте переменной. Критерии: 0 → 0, 1 → 1, 5 → 15, отрицательное число → понятное сообщение. Для результата используйте `long`.

04-B. Напишите приведённый сумматор по описанию, затем добавьте число успешно прочитанных значений. Критерии: вход `10/abc/-3/stop` даёт сумму 7, количество 2 и одно сообщение об ошибке; немедленный EOF — сумму 0 и количество 0. Знак `/` здесь разделяет строки, а не является вводом.

04-C. Ограничьте допустимое число диапазоном −1000…1000 и завершайте программу после трёх ошибочных строк подряд. Успешная строка сбрасывает счётчик ошибок. Критерии: `x/y/5/z/q/stop` не

останавливается раньше stop; `x/y/z/5` завершается до обработки 5.

Проверка понимания

Чем отличаются break и return? Когда тело while не выполнится ни разу? Почему пустая строка и EOF требуют разных решений? Как инвариант помогает проверить цикл?

Официальное описание: инструкции выбора и повторения.

Подсказки · Далее 05

05. Методы и декомпозиция

Результат: вы выделяете проверяемое вычисление из ввода-вывода и объясняете передачу аргументов. Нужна глава 04.

Контракт метода

Метод получает параметры, выполняет работу и возвращает результат. Контракт определяет допустимые входы и обещанный результат. Например, расчёт скидки принимает неотрицательную сумму и процент от 0 до 100. Пока вызывающая сторона проверяет эти условия через if.

Полный вариант в top-level `Program.cs`:

```
decimal result = CalculateTotal(125.50m, 4, 15m);
Console.WriteLine(result); // числовое значение 426.70

static decimal CalculateTotal(decimal price, int count, decimal discountPercent)
{
    decimal subtotal = price * count;
    decimal discounted = subtotal * (1 - discountPercent / 100m);
    return Math.Round(discounted, 2, MidpointRounding.ToEven);
}
```

В этом контексте объявлена локальная static-функция в сгенерированной точке входа. В классе похожее объявление является static-методом. `static` означает, что для вызова не нужен экземпляр класса; static локальная функция также не захватывает окружающие переменные.

Метод с типом результата `void` не возвращает значение. Оператор `return;` может досрочно завершить его. В методе с результатом все нормально завершающиеся пути должны вернуть значение.

Выделенное вычисление не читает Console и не пишет файл. Оно легче проверяется: одинаковые аргументы дают один результат. Вывод остаётся ответственностью вызывающего кода.

Параметр и аргумент

Параметр — имя в объявлении, аргумент — значение при вызове. Именованные аргументы делают понятнее несколько одинаково типизированных значений:

```
decimal total = CalculateTotal(price: 100m, count: 2, discountPercent: 10m);
```

Необязательный параметр задаёт значение по умолчанию в объявлении. Это значение подставляется при компиляции вызывающего кода: изменение default в библиотеке не обновляет уже скомпилированного клиента автоматически.

Перегрузка — несколько методов с одним именем и разными наборами параметров. Только возвращаемого типа для различения недостаточно. Для начальных упражнений предпочитайте разные понятные имена спорным перегрузкам.

`params` принимает переменное число аргументов. Классический пример:

```
static int Sum(params int[] values)
{
    int total = 0;
    foreach (int value in values)
    {
        total = checked(total + value);
    }
    return total;
}
```

`Sum(1, 2, 3)` вернёт 6, `Sum()` — 0. Массивы подробно разбираются в главе 07. Новые варианты params для коллекций изучаются после обобщений.

Передача по значению и по ссылке

По умолчанию копируется значение аргумента. Для int это число. Для ссылки на объект копируется ссылка; два имени могут указывать на один объект. Фраза «все объекты передаются по ссылке» скрывает важное различие между копированием ссылки и ref-параметром.

```
int number = 10;
Change(number);
Console.WriteLine(number); // 10
Increase(ref number);
Console.WriteLine(number); // 11

static void Change(int value) => value = 99;
static void Increase(ref int value) => value++;
```

`ref` даёт методу доступ к переменной вызывающей стороны. Она должна быть заранее инициализирована. `out` позволяет методу присвоить результат; он обязан присвоить его на всех нормально завершающихся путях. `in` предоставляет чтение аргумента по ссылке и не гарантирует отсутствие всех копий или ускорение. Начинайте с обычных параметров; модификаторы вводите ради понятного контракта.

Try-паттерн

Ожидаемо неверный текст удобно выражать результатом bool и out-значением:

```
static bool TryReadCount(string text, out int count)
{
    count = 0;
    if (!int.TryParse(text, out int parsed) || parsed < 1 || parsed > 100)
    {
        return false;
    }

    count = parsed;
    return true;
}
```

Вызывающий код сначала проверяет bool. Для `"12"` получит true и 12; для `"0"`, `"abc"`, `"101"` — false и 0. Парсинг ещё не означает валидацию: `int.TryParse("0", ...)` успешен, но правило количества запрещает ноль.

Рекурсия и размер метода

Рекурсивный метод вызывает себя и должен приближаться к базовому случаю. Для факториала `0! = 1`, а для n > 0 результат равен n × (n−1)!. Каждый незавершённый вызов требует места в стеке; очень глубокая рекурсия опасна. Для длинных линейных последовательностей цикл обычно проще. C# не обещает автоматическое устранение хвостовой рекурсии.

Хорошая граница метода — одна ответственность, которую можно назвать и проверить. Метод, который спрашивает путь, читает файл, считает скидки и печатает отчёт, связывает несколько независимых причин изменения.

Практика

05-A. Выделите `CalculateTotal` и вызовите его для трёх случаев: 100×2, скидка 10 → 180; 5×1, скидка 100 → 0; 0×3, скидка 0 → 0. Критерий: внутри метода нет Console.

05-B. Реализуйте `TryReadCount` без копирования. Критерии: `"1"` и `"100"` успешны, `"0"`, `"101"`, `"-2"`, `"x"` отклонены. При отказе out-параметр равен 0.

05-C. Создайте рекурсивный и циклический расчёт факториала с контрактом `0 <= n <= 20` и результатом long. Критерии: 0 → 1, 5 → 120, 20 → 2432902008176640000. Вход 21 отклоняется до вычисления. Объясните, почему checked не заменяет проверку контракта.

Проверка понимания

Почему изменение int-параметра не изменяет исходную переменную? Как отличается out от ref? Где проверять аргументы и где обрабатывать пользовательский текст? Почему метод без ввода-вывода проще тестировать?

Официальное описание параметров

Подсказки · Далее 06

06. Строки, null и культура

Результат: вы задаёте однозначный формат данных и обрабатываете отсутствие значения. Нужна глава 05.

Строка и символ

`string` — неизменяемая последовательность кодовых единиц UTF-16. Методы `Trim`, `Replace` и `ToUpperInvariant` возвращают строку; исходный объект не меняется.

```
string original = " food ";
string cleaned = original.Trim();
Console.WriteLine(original.Length); // 6
Console.WriteLine(cleaned.Length);  // 4
Console.WriteLine("\U0001F600".Length); // 2 кодовые единицы UTF-16, emoji
```

Один `char` не всегда соответствует одному видимому символу. `Rune` представляет Unicode scalar value, а составной видимый символ может включать несколько таких значений. Обрезание пользовательского текста по индексам char способно разрезать суррогатную пару. Для текстовых элементов существует `StringInfo`; практическая работа с Unicode будет в главе 26.

Запись и построение строк

```
string escaped = "first\nsecond";       // перевод строки
string path = @"C:\data\expenses.txt"; // обратные слэши буквально
string category = "food";
decimal amount = 12.50m;
string message = $"{category}: {amount:F2}";
string json = """{"category":"food","amount":12.50}""";
Console.WriteLine(message);
```

В строке с `$` выражения внутри фигурных скобок вычисляются. `:F2` задаёт два знака после десятичного разделителя, но не закрепляет культуру. Raw literal использует тройные кавычки; для многострочных raw strings важен отступ перед закрывающими кавычками.

Для нескольких фрагментов достаточно интерполяции. При накоплении большого текста в цикле `StringBuilder` позволяет уменьшить создание промежуточных строк. Не заменяйте все строки на StringBuilder без измерения и понятной причины.

Правила сравнения

Для команды, ключа протокола или категории из формата данных удобно явно выбрать:

```
bool same = string.Equals("FOOD", "food", StringComparison.OrdinalIgnoreCase);
Console.WriteLine(same); // True
```

`Ordinal` сравнивает кодовые единицы, `OrdinalIgnoreCase` дополнительно игнорирует регистр по соответствующим правилам .NET. Пользовательская алфавитная сортировка требует культурных правил. Сравнение строк нельзя универсально заменить вызовами `ToLower()`: это создаёт строки и скрывает выбор культуры. Подробнее: сравнение строк.

Число и его текстовое представление

У числа нет «запятой внутри». Разделитель относится к разбору и форматированию текста. Для машинного формата курса используем точку, запрет разделителей тысяч и явный InvariantCulture.

```
using System.Globalization;

bool ok = decimal.TryParse(
    "12.50",
    NumberStyles.AllowDecimalPoint,
    CultureInfo.InvariantCulture,
    out decimal amount);

Console.WriteLine(ok); // True
Console.WriteLine(amount.ToString("F2", CultureInfo.InvariantCulture)); // 12.50
```

Выбранные стили не разрешают знак, пробелы, экспоненту или разделители тысяч. Разрешённые формы определяются контрактом: `TryParse` не гарантирует ровно два знака после точки. Например, для строгой суммы в лабораторной мы отдельно проверим цифры и дробную часть.

В интерфейсе для человека можно выбрать его культуру и объяснить допустимый ввод. Нельзя незаметно использовать системную культуру для постоянного файла: перенос программы на другую машину тогда меняет смысл данных.

Отсутствие значения

`int?` — `Nullable<int>`: число либо отсутствие числа. `string?` — nullable-аннотация ссылочного типа для анализа компилятором. Она не создаёт новый CLR-тип строки и не вставляет автоматическую проверку во время выполнения. Подробнее: nullable reference types.

```
string? input = Console.ReadLine();
if (input is null)
{
    Console.WriteLine("Конец ввода");
    return;
}

string name = input.Trim();
Console.WriteLine(name.Length); // здесь input уже проверен
```

`null`, `""` и `"  "` — разные состояния. `string.IsNullOrWhiteSpace` объединяет их для задач, где ни одно не считается заполненным именем.

```
string? comment = null;
int length = comment?.Length ?? 0;
comment ??= "Без комментария";
Console.WriteLine(length);  // 0
Console.WriteLine(comment); // Без комментария
```

`?.` прекращает доступ, если слева null. `??` выбирает запасное значение при null, а `??=` присваивает его только при null. Оператор `!` после выражения лишь подавляет предупреждение компилятора. `comment!.Length` всё равно упадёт, если comment действительно null. Не устраняйте предупреждения массовым добавлением `!`.

Практика

06-A. Для `null`, пустой строки, трёх пробелов и `"  C#  "` определите, допустимо ли имя. Критерий: первые три отклонены, четвёртое преобразовано в `"C#"`. Различите завершение ввода и введённое пустое имя.

06-B. Напишите `TryReadAmount`: допустимы неотрицательные десятичные числа с точкой, без знака, пробелов и разделителя тысяч. Используйте явный NumberStyles. Критерии: `12.50` → 12.5m, `0` → 0m; `12,50`, `1,000`, `-1`, x — отказ. Вывод успешных сумм — F2 и InvariantCulture.

06-C. Уточните контракт B: от 1 до 999999.99 включительно, целая часть обязательна, после точки ровно одна или две цифры; форма без точки разрешена. Критерии: `1`, `1.5`, `1.50` успешны; `.5`, `1.`, `1.234`, `1000000` отклонены. Повторите проверки при CurrentCulture `ru-RU` и `en-US`: результат должен совпасть.

Проверка понимания

Почему Length не всегда считает видимые символы? Что именно гарантирует `string?`? Почему `!` не лечит NullReferenceException? Чем InvariantCulture отличается от Ordinal по назначению? Почему «парсится как decimal» слабее контракта собственного формата?

Подсказки · Далее 07

07. Массивы и базовые коллекции

Результат: вы храните набор значений, выбираете структуру под операцию и учитываете общие ссылки. Нужна глава 06.

Массив и индекс

Массив хранит фиксированное число элементов одного типа. Индексы начинаются с нуля; последний — `Length - 1`.

```
int[] values = [10, 20, 30]; // коллекционное выражение, C# 12+
Console.WriteLine(values[0]);  // 10
Console.WriteLine(values[^1]); // 30, первый с конца
int[] firstTwo = values[..2]; // новая копия [10, 20]
```

`new int[3]` создаёт три нуля. `new string[3]` создаёт три null, даже если тип элементов написан как string и nullable-анализ включён. Заполненность массива — отдельная ответственность. Индекс за границами приводит к исключению. Для пустого массива обращение к `[^1]` тоже ошибочно.

Прямоугольный массив объявляют `int[,] matrix = new int[2, 3];`. Ступенчатый `int[][] rows` — массив ссылок на массивы, строки могут иметь разную длину. Это разные модели: вложенный foreach и размеры нужно выбирать соответственно.

Копия ссылки и копия данных

```
int[] original = [1, 2];
int[] alias = original;
alias[0] = 9;
Console.WriteLine(original[0]); // 9

int[] copy = original[..];
copy[0] = 7;
Console.WriteLine(original[0]); // 9
```

Присваивание копирует ссылку на тот же массив. Срез массива создаёт новый массив. Если элементы сами ссылки на изменяемые объекты, копия массива всё ещё содержит ссылки на те же элементы: это поверхностное копирование. Для span семантика среза будет другой — представление той же памяти, глава 23.

Выбор коллекции

Структура	Для чего	Важные свойства
`List<T>`	Упорядоченный изменяемый список	Индекс O(1), поиск O(n), добавление в конец обычно O(1) амортизированно
`Dictionary<TKey,TValue>`	Поиск значения по ключу	В среднем O(1), уникальные ключи
`HashSet<T>`	Уникальные элементы	В среднем O(1) для поиска/добавления
`Queue<T>`	Первый пришёл — первый вышел	Enqueue/Dequeue
`Stack<T>`	Последний пришёл — первый вышел	Push/Pop

T — параметр типа. `List<decimal>` хранит decimal; детальное устройство обобщений будет позже. Оценка O(n) означает рост работы пропорционально размеру набора, а не конкретное число миллисекунд. Хеш-коллекции не гарантируют постоянное время для любого входа; качество хешей и сравнение важны.

```
List<decimal> amounts = [12.50m, 7.50m];
amounts.Add(3m);
decimal total = 0m;
foreach (decimal amount in amounts)
{
    total += amount;
}
Console.WriteLine(total); // числовое значение 23.00
```

`Count` — текущее число элементов списка, `Capacity` — ёмкость внутреннего хранения. Изменять состав списка при foreach обычно нельзя: перечислитель обнаруживает изменение. Для удаления используйте отдельный список результатов, RemoveAll или подходящий индексный цикл.

Группировка без LINQ

```
Dictionary<string, decimal> totals = new(StringComparer.OrdinalIgnoreCase);
string[] categories = ["food", "travel", "FOOD"];
decimal[] amounts = [12.50m, 30m, 7.50m];

for (int i = 0; i < categories.Length; i++)
{
    string category = categories[i];
    totals.TryGetValue(category, out decimal previous);
    totals[category] = previous + amounts[i];
}
Console.WriteLine(totals["food"]);   // числовое значение 20.00
Console.WriteLine(totals["travel"]); // числовое значение 30.00
```

При отсутствующем ключе TryGetValue возвращает false и default для значения. Для decimal это 0, что здесь удобно. Доступ `totals["missing"]` для чтения выбросит исключение; для записи индексатор добавляет или обновляет элемент. Add с уже существующим ключом — ошибка.

Не используйте порядок Dictionary как договор отчёта. Если результат должен воспроизводиться, извлеките ключи в List и вызовите `Sort(StringComparer.Ordinal)`, затем выводите по этому порядку. Параллельные массивы в примере требуют одинаковых длин; в следующей главе объединим связанные данные в объект.

Практика

07-A. Для массива `[3, 1, 4]` найдите сумму, минимум и максимум без LINQ. Критерии: 8, 1, 4. Для пустого массива сумма 0, а минимум/максимум явно отсутствуют; не выдавайте ноль за найденный минимум.

07-B. Реализуйте приведённую группировку самостоятельно. Критерии: food=20.00, travel=30.00; `FOOD` и `food` объединяются; вывод отсортирован. Добавьте четвёртую запись transport=0.01 и проверьте общую сумму 50.01.

07-C. Сделайте очередь из команд `add`, `report`, `quit` и обработайте её до quit. Сравните результат с теми же командами в Stack. Затем создайте HashSet идентификаторов `a, b, a`: критерий — два уникальных элемента. Объясните выбор структур словами.

Проверка понимания

Чем различаются Length и Count? Когда присваивание массива копирует элементы? Почему readonly-ссылка на List не обещает неизменность его содержимого? Почему нужен comparer и почему нельзя считать словарь автоматически отсортированным?

Официальный обзор коллекций

Подсказки · Далее 08

08. Классы и инварианты

Результат: вы объединяете связанные данные и поддерживаете допустимое состояние объекта. Нужна глава 07.

Тип и экземпляр

Класс описывает данные и действия. Экземпляр — отдельный объект этого класса. Если категория и сумма лежат в разных массивах, их длины и индексы легко перепутать. Объект Expense хранит их вместе.

Ниже полный тип для отдельного файла `Expense.cs`:

```
namespace Course;

public sealed class Expense
{
    public string Category { get; }
    public decimal Amount { get; }

    public Expense(string category, decimal amount)
    {
        if (string.IsNullOrWhiteSpace(category))
        {
            throw new ArgumentException("Категория обязательна.", nameof(category));
        }
        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(nameof(amount), "Сумма должна быть положительной.");
        }

        Category = category.Trim();
        Amount = amount;
    }
}
```

`throw` завершает нормальный ход создания объекта исключением; обработка — в главе 09. `nameof(amount)` получает имя параметра как строку и сохраняет связь при переименовании. После успешного конструктора всегда есть непустая категория и положительная сумма: это инвариант объекта. `sealed` запрещает наследование от класса; наследование рассмотрим позже.

В `Program.cs` того же проекта:

```
using Course;

Expense expense = new("food", 12.50m);
Console.WriteLine(expense.Category);
// expense.Amount = -1m; // не компилируется: доступного setter нет
```

Здесь два файла одного проекта, а не две самостоятельные программы. Запускаемый пример главы находится в проекте Foundations.

Поле, свойство и доступность

Поле хранит значение непосредственно. Свойство задаёт доступ через get/set. Авто-свойство позволяет компилятору создать backing field. Публичный set разрешает изменение; private set оставляет его внутри класса; get без set в этом примере задаётся при создании.

Модификатор	Доступ
`public`	Доступен потребителям типа

Модификатор	Доступ
`private`	Только внутри содержащего типа
`internal`	Внутри сборки
`protected`	В типе и производных типах
`protected internal`	В сборке или в производном типе
`private protected`	В производном типе в той же сборке

Тип верхнего уровня без модификатора обычно internal, члены класса — private. Вложенные типы имеют свои правила области видимости. Папка проекта не задаёт namespace автоматически.

`this` обозначает текущий экземпляр. `static`-поле принадлежит типу, поэтому все экземпляры наблюдают одно состояние. Такая общая изменяемая переменная усложняет тестирование и конкуренцию; не превращайте её в универсальное хранилище.

Неизменяемость и инициализация

`readonly`-поле можно присвоить при объявлении или в соответствующем конструкторе. Если оно содержит List, запрещено присвоить другой список; добавлять элементы в существующий список всё ещё можно. Неизменяемость ссылки не равна неизменяемости объекта.

`init` позволяет присваивание в фазе инициализации, а `required` требует указания члена при создании по правилам компилятора. Например, object initializer выглядит как `new Options { Title = "Report" }`. Ни init, ни required сами по себе не проверяют, что строка непустая, а число положительное.

Конструктор в нашем примере связывает проверку и создание. Внешнему коду нельзя предложить сначала `new Expense()`, затем отдельно «не забудьте вызвать Validate». Для типов с изменяемым состоянием методы должны сохранять инвариант после каждого действия.

Общие ссылки

```
List<Expense> first = [new("food", 12.50m)];
List<Expense> second = first;
second.Add(new("travel", 30m));
Console.WriteLine(first.Count); // 2
```

Списки first и second — два имени для одного объекта. Создание нового `List<Expense>(first)` копирует набор ссылок, но не создаёт новых Expense. В данном примере элементы предоставляют только чтение, поэтому совместное использование упрощается. Для изменяемого элемента оно требует осторожности.

Композиция означает, что один объект содержит другой и делегирует ему работу. Для журнала расходов естественнее хранить список Expense внутри Ledger, чем наследовать Ledger от List. Так можно разрешить только предметные операции и сохранять правила.

Практика

08-A. Создайте Expense в отдельном файле и два разных экземпляра. Критерий: изменение списка с одним объектом не создаёт автоматически второй объект; покажите совпадение ссылок через `ReferenceEquals` при присваивании.

08-B. Создайте `Product` с непустым Name и Price > 0, только get-свойства. Критерии: `("Book", 15m)` создаётся; пустое имя, пробелы, 0 и −1 отклоняются конструктором. Попытка присвоить Price снаружи не компилируется. Проверяйте такой контрпример в отдельной копии.

08-C. Создайте `Basket`, который содержит `private List<Product>`, метод Add и метод Total. Не выдавайте наружу изменяемый список. Критерий: два товара 15m и 7.5m дают 22.5m, пустая корзина — 0m. Объясните, какое правило проще поддерживать благодаря закрытому списку.

Проверка понимания

Почему public set может разрушить инвариант? Что гарантирует required и чего не гарантирует? Почему readonly List не становится immutable? Чем экземпляр отличается от типа и от ссылки?

Официальное описание классов

Подсказки · Далее 09

09. Ошибки, ресурсы и первые файлы

Результат: вы разделяете неверные данные и сбой операции, закрываете ресурсы и читаете текстовый файл. Нужна глава 08.

Когда возвращать результат, когда бросать исключение

Неправильная строка из пользовательского файла ожидаема: Try-метод сообщает об отказе и причине. Вызов конструктора с нарушенным контрактом — ошибка использования API: уместно исключение. Отказ файловой системы нельзя предсказать полной предварительной проверкой: его нужно обработать.

Исключение прерывает обычное выполнение и ищет подходящий catch по стеку вызовов. Если подходящего обработчика нет, процесс обычно завершится с ошибкой. Не используйте пустой catch: он превращает сбой в незаметно неверный результат.

```
try
{
    string text = File.ReadAllText("expenses.txt");
    Console.WriteLine(text);
}
catch (FileNotFoundException)
{
    Console.Error.WriteLine("Файл расходов не найден.");
}
catch (UnauthorizedAccessException)
{
    Console.Error.WriteLine("Недостаточно прав для чтения файла.");
}
catch (IOException ex)
{
    Console.Error.WriteLine($"Ошибка чтения: {ex.Message}");
}
```

Более конкретный catch пишут перед общим. Сообщение полезно человеку, но программа не должна разбирать локализованный текст Exception.Message для принятия решения. Данные для диагностики — тип, stack trace и контекст операции.

Повторное выбрасывание и finally

В catch запись `throw;` передаёт то же исключение выше, сохраняя исходный стек. `throw ex;` меняет информацию о месте повторного выбрасывания и затрудняет диагностику. Новое исключение с исходным в InnerException уместно, когда нужно выразить другой уровень абстракции.

`finally` выполняет завершающие действия при нормальном выходе и при обычном раскручивании стека. Это не гарантия при аварийном завершении процесса или отключении питания. Фильтр `catch (IOException ex) when (...)` принимает исключение только при истинном условии; не включайте в фильтр побочные действия, на которые рассчитывает программа.

Ресурс и сборщик мусора

Файл, сокет и подключение могут владеть ограниченным ресурсом ОС. GC управляет памятью; он не обещает немедленного закрытия файлов при выходе переменной из области видимости. `IDisposable.Dispose` выражает освобождение, а using обеспечивает вызов Dispose на выходе из блока.

```
using System.Text;

using (StreamReader reader = new(
    "expenses.txt",
    new UTF8Encoding(false, true),
    detectEncodingFromByteOrderMarks: false))
{
    string? line;
    while ((line = reader.ReadLine()) is not null)
    {
        Console.WriteLine(line);
    }
}
```

Здесь UTF-8 декодер отклоняет некорректные байты. Режим с BOM требует отдельного решения; контракт лабораторной допускает UTF-8 BOM и удаляет его явно. Форма `using var reader = ...;` освобождает ресурс в конце текущей области видимости. Она удобна, но большой метод может удерживать файл дольше, чем требуется.

Рабочая директория и путь

Относительный путь считается от рабочей директории процесса, которая не обязана совпадать с папкой исходника или исполняемого файла. Для диагностики полезен `Environment.CurrentDirectory`. Для составления пути используйте `Path.Combine(folder, "expenses.txt")`, для явного абсолютного представления — `Path.GetFullPath(path)`.

`File.Exists` не делает последующее чтение безопасным: между проверкой и открытием файл может исчезнуть, права могут измениться. Обрабатывайте ошибку самой операции. Не вставляйте произвольное пользовательское имя в доверенный путь без проверки границ: в веб-приложениях это особенно важно.

`File.ReadAllLines` загружает все строки в память. `File.ReadLines` перечисляет лениво: ошибка может возникнуть во время foreach, поэтому try должен охватывать перечисление, а не только создание последовательности.

Коды завершения

Обычный результат пишут в stdout, диагностику — в stderr. В top-level программе `return 0;` сообщает успех, `return 1;` — ошибку по договору приложения. Разным категориям ошибок можно назначить разные коды. Это позволяет автоматизации понять результат, не разбирая текст сообщения.

Не печатайте итоговый успешный отчёт после частичной ошибки, если контракт требует обработать файл целиком. Сначала прочитайте и проверьте данные, затем формируйте результат.

Практика

09-A. Создайте UTF-8 файл из трёх строк, прочитайте его using+StreamReader. Критерий: ровно три строки в исходном порядке; после выхода из using файл можно снова открыть.

09-B. Напишите утилиту подсчёта непустых строк. Контракт: путь — один аргумент, успех → число в stdout и код 0; ошибка аргументов → stderr и код 2; ошибка чтения → stderr и код 3. Критерии: пустой файл → 0, файл `a/пустая строка/b` → 2, отсутствующий файл → код 3 без результата в stdout.

09-C. Добавьте strict UTF-8 и приём файла с неверным байтом. Он должен завершаться диагностикой декодирования, не подменяя байт символом замены. Продемонстрируйте освобождение ресурса при исключении внутри блока чтения. Критерий: повторное открытие файла после catch возможно.

Проверка понимания

Что освобождает GC и что освобождает Dispose? Чем throw отличается от throw ex? Почему File.Exists не исключает гонку? Почему try вокруг создания File.ReadLines недостаточен? Что увидит вызывающий скрипт при записи ошибки в stdout вместо stderr?

Обработка исключений · using

Подсказки · Далее 10

10. Практикум ExpenseLedger CLI

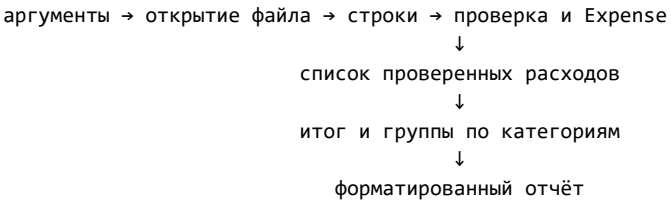
Результат: готовая консольная программа читает расходы, проверяет формат и выводит отчёт. Нужны главы 01–09. Подробный договор — в лабораторной L01.

Постановка задачи

Пользователь хранит небольшую выгрузку расходов в UTF-8 файле. Ему нужны число записей, сумма и суммы по категориям. Формат не должен зависеть от языка Windows. В ошибочной выгрузке нужно указать номер строки, чтобы её можно было исправить. При ошибке данных успешный отчёт не выдаётся.

Это первая объединённая работа: она требует почти всех уже изученных приёмов. Сначала реализуйте её в `work/ExpenseLedger`. Полный пример в `examples/ExpenseLedger` используйте для сравнения после собственной попытки.

Разделение ответственности



У программы четыре границы: аргументы процесса, файловая система, текстовый формат и модель данных. Каждая требует своего решения. Неверный decimal нельзя превращать в 0: это создаст фиктивный расход. Нельзя также складывать данные до проверки всех строк и печатать частичный результат как полный.

Разумный первый набор методов: `TryParseLine`, `TryParseAmount`, `ReadAll`, `WriteReport`. Два первых отвечают за данные, следующие — за I/O. Класс Expense сохраняет инварианты. Сложные generic-абстракции на этом этапе не нужны.

Шаги реализации

1. Создайте консольный проект net10.0, включите nullable и убедитесь, что он запускается.
2. Обработайте отсутствие пути и лишние аргументы. Проверьте код завершения.
3. Напишите разбор суммы на отдельных строках. Проверьте границы до чтения файла.
4. Разберите строку на три поля и проверьте дату, категорию и сумму.
5. Создайте тип Expense и запретите недопустимое состояние.
6. Прочитайте файл, сохранив физические номера строк, включая пустые.
7. Посчитайте итог и группы через Dictionary. Отсортируйте ключи явно.
8. Добавьте обработку ошибок чтения, строгую кодировку и раздельные stdout/stderr.
9. Пройдите все сценарии приёмки и напишите короткую инструкцию запуска.

Даты используют новый для вас тип `DateOnly` из BCL: он хранит календарную дату без времени и зоны. `TryParseExact` с шаблоном `yyyy-MM-dd` проверяет реальную дату, например отклоняет 2026-02-30. Не пытайтесь проверить календарь только количеством символов и тире.

Разобранный алгоритм отчёта

Для каждой записи увеличьте общее число и общую сумму. Получите предыдущую сумму категории через TryGetValue и запишите новую. Категории по договору нечувствительны к регистру; нормализуйте их к нижнему ASCII-регистру и используйте Ordinal для ключей.

После окончания расчёта создайте список ключей словаря, отсортируйте его и выведите значения в F2 и InvariantCulture. Сумма для входа из примера — 50.00 независимо от CurrentCulture. Приведение регистра допустимо здесь потому, что формат категорий ограничен ASCII и результат должен иметь каноническое написание. Для произвольных пользовательских строк правила шире.

Проверка решения

Проверьте success, пустой файл, повтор категории с разным регистром, несуществующий файл, невозможную дату, отрицательную сумму, лишнее поле и неправильную кодировку. Для каждого случая проверяются три наблюдаемых результата: stdout, stderr и exit code.

Готовый пример запускается так:

```
dotnet run --project examples/ExpenseLedger -- labs/01-expense-ledger/data/valid.txt
python tools/verify.py
```

Проверка собственной реализации с тем же контрактом:

```
dotnet build work/ExpenseLedger -c Release
python tools/verify.py --ledger-dll work/ExpenseLedger/bin/Release/net10.0/ExpenseLedger.dll
```

Скрипт использует временные тестовые файлы и сверяет поведение. Автоматические проверки помогают найти нарушение контракта, но не доказывают качество всех решений. Перед сдачей самостоятельно объясните выбор типов, условия ветвлений и область жизни reader.

Практика

10-A. Реализуйте основной сценарий L01. Критерий: точное совпадение отчёта из лабораторной.

10-B. Реализуйте весь контракт и пройдите сценарии ошибок. Критерии: при любом ошибочном файле stdout пуст, причина в stderr, код соответствует категории сбоя. После ошибки в строке 2 программа не должна печатать итог строки 1.

10-C. В собственной ветке добавьте необязательный фильтр месяца `yyyy-MM`. Сначала отдельно запишите новый контракт аргументов. Полный файл всё равно валидируется, затем фильтруются записи. Критерии: месяц без записей → пустой отчёт; ошибочная строка вне выбранного месяца всё равно отклоняется. Базовые проверки B должны проходить при запуске без фильтра. Готовый пример реализует только A/B.

Допуск к следующему блоку

Объясните без запуска, что произойдёт с `1,50`, `2026-02-30`, `FOOD`, UTF-16 файлом и EOF в пустом файле. Покажите в коде единственное место определения допустимого формата суммы. Попросите другого человека запустить программу по вашей инструкции в новой папке.

После сдачи переходите к главам 11–18 из программы: интерфейсы, равенство, обобщения, делегаты, LINQ и библиотека предметной области. Их учебный текст ещё не подготовлен; состояние наполнения находится в PROGRESS.

Подсказки · Лабораторная

L01. Консольный журнал расходов

Реализуйте программу ExpenseLedger по этому контракту. Предварительно пройдите главы 01–09 и разбор проекта. Цель — связать язык и BCL в работающую утилиту. Ориентир: 6–10 часов, включая самостоятельные проверки и исправления.

Входные данные

Принимается ровно один аргумент: путь к файлу. Файл — UTF-8, необязательный UTF-8 BOM допустим. UTF-16 и повреждённый UTF-8 отклоняются. Строки заканчиваются LF или CRLF; последняя может не иметь перевода строки. Пустые строки и строки из пробелов пропускаются. Номер ошибочной строки считается по физическим строкам файла, включая пропущенные. Заголовка и комментариев в формате нет.

Каждая непустая строка имеет ровно три поля:

```
дата;категория;сумма
2026-01-10;food;12.50
```

Пробелы по краям каждого поля удаляются перед проверкой. Внутренние пробелы не удаляются.

Поле	Правило
Дата	Ровно `yyyy-MM-dd`, настоящая календарная дата в диапазоне DateOnly
Категория	1–32 ASCII-символа; первый — буква; остальные — буквы, цифры, `_`, `-`
Сумма	Только ASCII-цифры и необязательная точка; целая часть обязательна; после точки 1–2 цифры
Диапазон суммы	Больше 0 и не больше 999999.99
Число записей	Не больше 100000 непустых корректных строк

Разрешены `1`, `1.5`, `1.50`, `001.50`; запрещены `+1`, `-1`, `0`, `.5`, `1.`, `1,50`, `1e2`, `1.234`, `NaN`, разделители тысяч. Регистры категорий не различаются; в результате выводится нижний ASCII-регистр. Повторные строки считаются отдельными расходами: правило дедупликации здесь не задано.

Лимит записей позволяет держать первый учебный вариант в памяти. Это формат небольшой локальной выгрузки; потоковый импорт и лимиты байтов изучаются в следующих блоках.

Результат

Сначала выведите count и total, затем категории в порядке StringComparer.Ordinal. Суммы всегда имеют точку и два знака после неё. Культура компьютера не влияет на результат. Успешный stderr пуст; порядок строк определён контрактом.

Для valid.txt:

```
count=3
total=50.00
food=20.00
travel=30.00
```

Для пустого файла:

```
count=0
total=0.00
```

Ошибки и коды завершения

Код	Ситуация	Требование к stderr
0	Успех	Пуст
2	Не один аргумент	Начинается с `usage:`
3	Ошибка пути, открытия или чтения	Начинается с `io:`
4	Ошибка данных или кодировки	`line N: причина` либо `encoding: причина`

При любом ненулевом коде stdout пуст. Для ошибки данных достаточно сообщить первую; номер строки обязателен. Текст причины после префикса может быть русским или английским. При повреждении кодировки номер строки не требуется, поскольку буферизация чтения усложняет его определение.

У invalid.txt ошибочна дата в строке 2. Код — 4. Строка 1 не должна приводить к частичному успешному отчёту.

Требования к реализации

- Суммы представлены decimal; при приёме данных точность проверяется, а не молча округляется.
- Есть отдельный тип расхода и методы разбора без чтения Console или файла.
- После успешного создания расход удовлетворяет правилам категории и суммы.
- Файл освобождается при успехе, ошибке данных и ошибке чтения.
- Для группировки используется Dictionary, для порядка вывода — явная сортировка.
- Нет подавления nullable-предупреждений с помощью ! без объяснённого инварианта.
- При максимальном количестве записей расчёт остаётся в допустимом диапазоне decimal.

Минимальные сценарии приёмки

Вход или изменение	Ожидание
Основной файл	Точный отчёт выше
Пустой файл и только пробелы	count=0, total=0.00
`food` и `FOOD`	Одна категория
Две одинаковые строки	Два расхода
Сумма 0.01 и 999999.99	Обе принимаются
0, −1, 1000000, 1.234, 1,50	Код 4, номер строки
2024-02-29 / 2026-02-29	Первая дата допустима, вторая нет
Пустая категория, 33 символа, буква вне ASCII	Код 4
Четвёртое поле	Код 4
Пустая строка перед ошибочной	Учитывается в номере ошибки

Вход или изменение	Ожидание
100000 / 100001 записей	Успех / код 4
UTF-8 BOM / неверный байт UTF-8	Успех / код 4
Несуществующий файл	Код 3
Нет аргумента / два аргумента	Код 2
Разные системные культуры	Одинаковый отчёт

Как проверить и сдать

Соберите свою программу в Release. Выполните:

```
python tools/verify.py --ledger-dll work/ExpenseLedger/bin/Release/net10.0/ExpenseLedger.dll
```

Без параметра скрипт проверяет готовые примеры курса и их сборку. Подготовьте исходники, команду запуска и запись трёх найденных вами дефектов с причиной. Минимум для зачёта: контракт выполнен полностью, все проверки пройдены, код объясним без образца.

Оценка из 10: корректность 4; граничные случаи и ошибки 2; декомпозиция и инварианты 2; воспроизводимость и ясность 2. Даже при хорошей сумме баллов ошибка денежного расчёта, частичный отчёт при сбое или неосвобождённый ресурс требуют исправления.

Дополнительное изменение

После зачёта добавьте фильтр месяца из задачи 10-C. Отдельно запишите контракт, проверьте пустой результат и ошибочную строку вне выбранного месяца. Сравните архитектуру: удалось ли добавить фильтр, не переписывая парсер суммы?

Подсказки и разбор контрольных вопросов

Здесь намеренно даны направления решения и объяснения, а не готовые программы для всех задач. Открывайте соответствующий пункт после самостоятельной попытки. Полные учебные примеры находятся в examples, эталон L01 — в ExpenseLedger.

Главы 01 и 02

01-B: `--no-build` запускает уже собранный результат. Изменённый исходник сам не становится новой сборкой. 01-C: DLL находится в publish-каталоге и запускается подходящим runtime; файл исходника для этого не нужен.

02-B: неизвестное имя обнаруживает компилятор; целочисленное деление на ноль при переменной-делителе — среда выполнения; замена умножения сложением может быть полностью допустимым кодом. 02-C: условие breakpoint проверяется в точке остановки, где total уже должен быть вычислен.

SDK включает инструменты сборки, runtime исполняет код. Namespace не устанавливает пакет. Область видимости локального bonus заканчивается вместе с блоком. Диагностика nullable — не замена проверке внешних данных.

Глава 03

03-A: `7 / 3` даёт int 2; `7 / 3m` даёт decimal с конечным приближением к 2⅓; `-7 % 3` равно −1; cast 2.9 к int даёт 2. 03-B: 125.50 × 4 = 502.00; после скидки остаётся 85%, результат 426.70. 03-C: сначала приведите один множитель к long. Присваивание готового переполненного int в long не восстанавливает потерянное значение. checked должен охватывать именно умножение.

`var` выбирает один статический тип при компиляции. Decimal тоже не является бесконечно точной дробью. Правило округления принадлежит предметной задаче; его нельзя заменять произвольным приведением.

Глава 04

04-A: проверьте отрицательное n до цикла. При n=0 цикл от 1 до n не выполняется. 04-B: количество увеличивается только после успешного TryParse. 04-C: храните число последовательных ошибок; сбрасывайте после принятого значения. Выход через break заканчивает ближайший цикл, через return — весь метод. EOF приходит как null, Enter без текста — пустая строка.

Глава 05

05-A: метод должен возвращать число; форматирование делается отдельно. 05-B: заранее присвойте out-параметру 0. Это упрощает все пути отказа. 05-C: рекурсия имеет базовый случай n=0. Невалидный вход отклоняйте до первого рекурсивного вызова. 20! помещается в long, 21! — нет. Цикл не создаёт цепочку рекурсивных вызовов.

Обычный параметр копирует своё значение. Если значение — ссылка, копируется ссылка на общий объект. Ref позволяет менять переменную вызывающего кода, а out обязан получить значение от метода.

Глава 06

06-A: сначала различите null и обычную строку, затем проверьте IsNullOrWhiteSpace. 06-B: NumberStyles.AllowDecimalPoint совместно с InvariantCulture не разрешает запятую или знак. 06-C: проверка диапазона и количества цифр выполняется дополнительно к TryParse; проверяйте ASCII-цифры явно, если таков договор формата.

Для контрольных вопросов: Length считает кодовые единицы UTF-16; ! меняет анализ компилятора, но не объект во время выполнения. Ordinal определяет сравнение текста, CultureInfo — в том числе форматирование чисел. Эти инструменты решают разные задачи.

Глава 07

07-A: минимум можно инициализировать первым элементом только после проверки непустого массива. Либо используйте int? и явно представьте отсутствие результата. 07-B: comparer задаётся при создании Dictionary. Числовой итог 50.01 не зависит от порядка перечисления. 07-C: Queue вернёт add, report, quit; Stack начнёт с quit, и обработчик может сразу закончиться.

Срез массива создаёт новый контейнер. Если элементы — ссылки, сами объекты остаются общими. Словарь обеспечивает поиск по ключу; требуемый порядок отчёта задавайте сортировкой.

Глава 08

08-A: `Expense second = first;` копирует ссылку. `new Expense(...)` создаёт новый экземпляр. 08-B: get-only свойства и проверка конструктора не позволяют последующим присваиванием нарушить правило. 08-C: Basket содержит список и выдаёт только нужные операции. Внешний код не должен иметь возможность добавить недопустимый элемент в обход этих операций.

Required требует инициализации по правилам компилятора, но не обещает непустую строку. Readonly не делает изменяемый объект глубоко неизменяемым.

Глава 09

09-B: проверяйте число аргументов до открытия файла. Печатайте результат после завершения чтения. 09-C: `new UTF8Encoding(false, true)` включает исключение при неверных последовательностях байтов. Включение автоматического определения BOM может принять UTF-16; для строгого UTF-8 его отключают.

Using вызывает Dispose при обычном выходе и раскручивании исключения. Throw без аргумента сохраняет исходную цепочку диагностики. Между Exists и открытием файловая система может измениться. Ленивая последовательность способна выбросить исключение при MoveNext внутри foreach.

Глава 10 и L01

Сначала зафиксируйте формат и тестовые строки. Разбор суммы должен иметь один источник правил. Дата проверяется календарным типом, а не только разделением строки на части. Полностью прочитанный корректный набор отделяется от вывода — тогда ошибка строки 2 не выдаст отчёт строки 1.

Для 10-C сначала валидируйте весь вход, потом отбирайте месяц. Иначе фильтр случайно скрывает ошибочные записи и нарушает контракт. Не меняйте денежный расчёт при добавлении фильтра.

Эталонное решение — один из возможных вариантов. После сравнения закройте его и повторно реализуйте самый трудный метод по собственным тестам.

Карта покрытия языка C#

Карта помогает проверять полноту программы. Номер означает запланированное место темы, а статус показывает наличие учебного текста. **Написано только 01–10**; строка «план» не означает, что тему уже можно изучить здесь. Сверка состава выполнена с официальным справочником и описанием C# 14.

Область	Где изучается	Состояние
Лексика, литералы, комментарии, имена, scope	02–03	Базовый материал написан
Числовые типы, bool/char/string, conversions, checked	03, 06	Базовый материал написан
Операции, precedence, short-circuit, биты и сдвиги	03	Введение написано
if/switch, циклы, break/continue/return, goto	04, 17	04 написана, углубление в плане
Методы, static, overload, local functions, recursion, params	05, 13, 24	Основы написаны
ref/out/in, ref return, ref readonly, scoped	05, 23	Основы написаны, углубление в плане
Null, `Nullable<T>`, аннотации и анализ	06, 13, 21, 24	Основы написаны
Массивы, indices/ranges, collection expressions	07, 23–25	Основы написаны
class, constructors, fields, properties, accessibility	08, 11–12, 24	Основы написаны
init, required, readonly, object initializer	08, 12	Введение написано
Исключения, filters, using и IDisposable	09, 20, 31	Основы написаны
Наследование, override/new, abstract, sealed, interfaces	11	План
struct/readonly struct, records, primary constructors, with	12, 24	План
object, boxing/unboxing, enum/Flags, tuples, equality	12, 20	План
Generics, constraints, variance, generic math	13	План
delegate, lambda, closures, events	14	План
foreach pattern, IEnumerable, yield, iterator disposal	15	План
LINQ method/query syntax, IQueryable	16	План
Patterns, extension members, operators, indexers	17, 24	План
Assemblies, IL, metadata, CLR, JIT/AOT	01, 19	Обзор написан, углубление в плане
GC, finalization, managed/native lifetime	20	План
Attributes, reflection, dynamic, anonymous types, expressions	21	План

Область	Где изучается	Состояние
namespaces, aliases, global/static using, extern alias, file types	02, 22	Основы написаны, углубление в плане
partial, preprocessor, XML docs, Roslyn и source generators	22	План
Span/Memory, ref struct, unsafe, fixed, stackalloc, pointers	23	План
Function pointers, calling conventions, inline arrays, sizeof	23, 55	План
C# 13–14 и совместимость версий	24	План
async/await, awaitables, Task/ValueTask, async streams	31	План
lock, memory model, volatile, Interlocked	32	План
Interop, extern methods, marshaling, native ownership	55	План

Что находится за пределами синтаксиса

Работа с JSON, датами, файлами, HTTP, DI и коллекциями BCL изучается в 25–30. Проверки, проектирование и инструменты — в 36–39; SQL и persistence — в 40–43; веб и сервисы — в 44–49; UI, игры и вычисления — в 50–54. Поставка и эксплуатация — в 55–60.

Таким образом, «знать язык» и «работать в C#-проекте» проверяются отдельно. Редкие средства, например COM или unsafe, нужны не каждой роли, но для них задан явный маршрут. Карта охватывает семейства возможностей; точные ограничения грамматики и комбинации модификаторов проверяются по языковой спецификации и компилятору.

Как дополнять карту

При добавлении новой главы обновите её статус и проверьте, что тема имеет объяснение, пример, самостоятельное упражнение и критерий проверки. Если материал только упомянут в списке, статус остаётся «план». Новая глава обязательно входит в book.json и общий PDF.

Источники и версии

База курса — C# 14 и .NET 10. Дата сверки: 22 октября 2026. Учебные объяснения и задания написаны самостоятельно; книги используются как дополнительное чтение. Содержимое папки Books не включается в PDF курса.

Официальные источники

Назначение	Источник
Синтаксис и семантика	C# language reference
Нормативная модель и feature specifications	C# specification
Версия языка	C# 14
Поддержка SDK/runtime	.NET support policy
Работа с проектами	.NET CLI
Стандартная библиотека	.NET API browser
Nullable-анализ	Nullable reference types
Сравнение строк	String best practices
Веб-разработка	ASP.NET Core
ORM	Entity Framework Core

Справочник языка обновляется и может упоминать preview-возможности; в курс их включают только с явной пометкой. Существование API в документации не подтверждает его доступность в выбранном TFM. Запускаемые примеры компилируются под net10.0 и LangVersion 14.0.

Книги в репозитории

Книга	Файл в Books	Для чего использовать
Adam Freeman, Pro ASP.NET Core 6, 9th edition, 2022	`Adam_Freeman_Pro_ASP_NET_Core_6_Develop_Cloud_Ready_Web_Applications.pdf`	HTTP pipeline, MVC, Razor Pages, Blazor; главы 44–51
Xiaodi Yan, Web API Development with ASP.NET Core 8, 2024	`Xiaodi_Yan_Web_API_Development_with_ASP_NET_Core_8_Learn_techniques.pdf`	Проектирование и проверка API; главы 44–49
Jeffrey Richter, CLR via C#, русское издание	`Мастер_класс_Джеффри_Рихтер_CLR_via_C#_Программирование_на_платформе.pdf`	Типы, CLR, ресурсы и конкурентность; главы 11–24, 31–32

В первых двух файлах сверены титульные страницы. У файла Рихтера начальные страницы представлены без извлекаемого текста; номер издания здесь не утверждается. Вместо ненадёжных номеров PDF-страниц ищите соответствующую тему в оглавлении своего издания. Для начала курса книги не обязательны.

В старых книгах встречаются API и настройки эпохи .NET Framework, ASP.NET Core 6 или 8. Изучайте объяснение задачи, затем проверяйте современный способ по документации. Например, рекомендации о hosting model, nullable, пакетах и версиях SDK нельзя переносить механически.

Внешние библиотеки следующих блоков

Версии внешних NuGet-пакетов пока не зафиксированы, потому что главы с их примерами ещё не написаны. При наполнении каждой главы выбирается совместимая стабильная версия, выполняется restore/build/test и приводится ссылка на официальную документацию пакета. Текущий первый блок не использует внешние NuGet-пакеты.

Инструменты книги

PDF собирается из Markdown с помощью markdown-it-py и ReportLab. Кириллические TrueType-шрифты встраиваются в PDF. Проверка структуры использует pypdf. Основа структуры и сборщика адаптирована из соседнего курса SystemVerilog пользователя; учебный текст и примеры здесь относятся к C#.

Программа практикума C# и .NET

Программа ведёт от первой строки кода к самостоятельной разработке и сопровождению приложения. Базовая среда — **C# 14, .NET 10 LTS**, проверка актуальности: **22 октября 2026**. Версия языка и поддержка платформы проверяются отдельно.

Как читать программу

Ниже сначала перечислен весь маршрут, включая темы будущих выпусков. **Наполнены главы 01–10**; главы 11–60 пока имеют статус **план**. Их наличие здесь не означает готовность учебного текста. Оглавление написанных глав и порядок наполнения показывают фактическое состояние.

Общее ядро: 01–49 и 55–60; прикладная ветка: одна или несколько из 50–54. Знание синтаксиса само по себе не заменяет SQL, HTTP, тестирование и эксплуатацию: эти темы включены в маршрут. Для первого трудоустройства нужно глубоко пройти выбранную ветку, а остальные знать на уровне назначения и ограничений.

Оценки времени включают самостоятельное решение и исправления, а не только чтение. Весь маршрут с одной веткой потребует нескольких сотен часов. При опыте другого языка разрешается проходить входные проверки вместо повторного чтения; пропускать практику и контрольные работы нельзя. Скорость определяется прохождением проверок.

Два обязательных способа изучения

1. **Репозиторий как самостоятельный учебник:** тематические каталоги, оглавление, переходы между главами, ссылки на задания и запускаемые примеры. Каждую главу можно читать отдельно в Markdown.
2. **Весь учебный материал как один PDF:** все написанные главы, лабораторные и приложения собираются в `dist/csharp.pdf` по единому порядку `book.json`. По мере наполнения в эту же сборку войдут все 60 глав. PDF содержит оглавление, закладки, номера страниц, кириллицу, читаемый код, таблицы и внутренние ссылки.

Исходник текста один — Markdown в репозитории; отдельная вручную поддерживаемая версия PDF не создаётся. Команда сборки: `python tools/book.py build`. Глава считается готовой только после включения в оглавление, проверки ссылок, сборки общего PDF и проверки читаемости её страниц. Лабораторные условия тоже входят в PDF; исходный код проектов остаётся запускаемым в репозитории. Инструкция — в README.

Состав каждой главы

1. Результат и необходимые предыдущие знания.
2. Объяснение модели, затем синтаксис и небольшой разобранный пример.
3. Типичные ошибки, граничные случаи и способы диагностики.
4. Практика A — воспроизведение; B — самостоятельное решение; C — изменение требований.
5. Проверяемые условия готовности, вопросы и подсказки отдельно от задания.
6. Ссылки на первоисточники и связь со сквозным проектом.

Задачи имеют обозначения вида `03-A`; лабораторные — `L01`. В начале допускаются ручные сценарии; после главы 36 для нового поведения обязательны автоматические проверки. Готовый пример открывают после самостоятельной попытки. Ошибочный код всегда явно помечается как упражнение или

контрпример.

I. Первая самостоятельная программа

01–10; вход без опыта программирования; ориентир 30–45 часов.

01. C# и .NET, рабочая среда

Статус: написано.

1. Язык, компилятор Roslyn, IL, CLR, JIT и библиотеки.
2. SDK и runtime.
3. .NET и .NET Framework.
4. установка SDK и редактора.
5. CLI, csproj, slnx, TFM.
6. build, run, restore, publish.

Практический результат: Собрать и запустить приложение из терминала и отладчика.

02. Синтаксис и диагностика

Статус: написано.

1. Токены, идентификаторы, ключевые слова, литералы и комментарии.
2. выражение и инструкция.
3. блоки и область видимости.
4. top-level statements и Main.
5. using и пространства имён.
6. ошибки компиляции, исключения и логические ошибки.
7. breakpoint, step, watch, call stack.

Практический результат: Исправить три разных вида ошибок и объяснить каждый диагноз.

03. Типы, числа и операции

Статус: написано.

1. bool, char, целые знаковые и беззнаковые типы, nint/nuint, float/double/decimal.
2. литералы и суффиксы.
3. var, const, default.
4. преобразования.
5. checked/unchecked.
6. арифметика, сравнения, логика, биты и сдвиги.
7. приоритет и порядок вычисления.

Практический результат: Рассчитать стоимость заказа и проверить округление и границы.

04. Ветвления и циклы

Статус: написано.

1. if/else.
2. switch statement и expression.
3. for, while, do, foreach.
4. break, continue, return.
5. короткое замыкание.
6. интервалы и инвариант цикла.
7. EOF.
8. трассировка и граничные случаи.
9. область применения goto.

Практический результат: Сделать интерактивный сумматор с устойчивым вводом.

05. Методы и декомпозиция

Статус: написано.

1. Параметры и аргументы.
2. возвращаемые значения и void.
3. static.
4. перегрузки.
5. именованные и необязательные аргументы.
6. params.
7. локальные функции.
8. передача по значению, ref/out/in.
9. рекурсия и стек.
10. чистые вычисления и побочные эффекты.

Практический результат: Разделить расчёт, разбор данных и вывод; реализовать Try-метод.

06. Строки, null и культура

Статус: написано.

1. string и UTF-16.
2. char, Rune и текстовый элемент.
3. неизменяемость.
4. escape, verbatim, raw и interpolation.
5. StringBuilder.
6. сравнение Ordinal и культурное сравнение.

7. Parse/TryParse, CultureInfo.
8. nullable value/reference types, ?. и ??, ??= и !.

Практический результат: Определить строгий формат суммы и корректно обработать пустой ввод.

07. Массивы и базовые коллекции

Статус: написано.

1. Одномерные, прямоугольные и ступенчатые массивы.
2. индексы и диапазоны.
3. копирование и общие ссылки.
4. List, Dictionary, HashSet, Queue, Stack.
5. коллекционные выражения.
6. TryGetValue.
7. сложность O(n) и O(1).
8. перебор и изменение.

Практический результат: Сгруппировать расходы по категориям без LINQ.

08. Классы и инварианты

Статус: написано.

1. class, объект и ссылка.
2. поля, свойства, методы и конструкторы.
3. this.
4. public/private/internal.
5. readonly и static.
6. авто-свойства, get/set/init.
7. object initializer.
8. required.
9. namespace и файл.
10. инкапсуляция и композиция.

Практический результат: Создать объект расхода, который нельзя оставить в недопустимом состоянии.

09. Ошибки, ресурсы и первые файлы

Статус: написано.

1. Exception.
2. try/catch/finally, throw и throw ex.
3. catch filters.
4. Try-паттерн.

5. IDisposable и using.
6. Path, File, StreamReader/Writer, UTF-8.
7. относительные пути и рабочая директория.
8. обработка I/O на границе приложения.

Практический результат: Прочитать текстовый файл, сообщить о повреждённой строке и закрыть ресурс.

10. Практикум ExpenseLedger CLI

Статус: написано.

1. Контракт входного файла.
2. разбор строк.
3. модель предметной области.
4. список расходов и агрегаты.
5. протокол ошибок и exit codes.
6. детерминированный вывод.
7. ручные проверки, автоматические сценарии и ревью.

Практический результат: Поставить консольный отчёт по расходам с проверяемыми критериями приёмки.

II. Система типов и выразительный код

11–18; нужны 01–10; ориентир 35–50 часов.

11. Наследование и интерфейсы

Статус: план.

1. Базовый класс, base, abstract, virtual/override, sealed, new hiding.
2. полиморфизм.
3. интерфейсы и явная реализация.
4. default interface members.
5. static abstract/virtual members.
6. ковариантный результат.
7. композиция и подстановка.

Практический результат: Заменять хранилище расходов через интерфейс без изменения расчёта.

12. Структуры, записи и равенство

Статус: план.

1. struct, readonly struct, record class/struct.
2. primary constructors.
3. value/reference semantics.

4. boxing/unboxing.
5. with и поверхностное копирование.
6. Equals, GetHashCode, IEquatable, операторы.
7. enum и Flags.
8. ValueTuple и деконструкция.
9. ключи словаря.

Практический результат: Создать тип Money, определить равенство и проверить ключи коллекций.

13. Обобщения и контракты типов

Статус: план.

1. Generic типы и методы.
2. вывод типов.
3. where class/struct/notnull/unmanaged/new/base/interface/default.
4. allows ref struct.
5. variance in/out.
6. generic static state.
7. ограничения и static abstract для generic math.
8. nullable T.
9. обобщения и отражение.

Практический результат: Реализовать `Result<T>` и типобезопасный алгоритм без dynamic.

14. Делегаты, лямбды и события

Статус: план.

1. delegate, Action, Func, Predicate.
2. method group.
3. multicast и исключения.
4. анонимные функции.
5. замыкания и lifetime.
6. static lambdas.
7. event, подписка и отписка.
8. `EventHandler<T>`.
9. утечки через события.
10. async void в обработчиках.

Практический результат: Создать уведомления об изменениях и доказать освобождение подписчика.

15. Перечисление и итераторы

Статус: план.

1. IEnumerable/IEnumerator и generic варианты.
2. foreach pattern.
3. yield return/break.
4. отложенное выполнение.
5. повторное перечисление.
6. освобождение итератора.
7. lazy и eager.
8. бесконечные последовательности.
9. основы IAsyncEnumerable.

Практический результат: Сделать потоковый импорт большого файла без загрузки всего содержимого.

16. LINQ в памяти и в запросах

Статус: план.

1. Where/Select/SelectMany.
2. OrderBy/ThenBy.
3. GroupBy, Join/GroupJoin.
4. Any/All, First/Single и Default.
5. Aggregate, Count, Sum.
6. DistinctBy, Chunk.
7. query syntax.
8. materialization.
9. IEnumerable против IQueryable.
10. expression tree и перевод запроса.

Практический результат: Написать отчёты и найти ошибку из-за повторного перечисления.

17. Шаблоны и расширение синтаксиса

Статус: план.

1. is/as и pattern matching.
2. type, constant, property, positional, relational, logical, list и slice patterns.
3. guards и исчерпываемость.
4. extension methods/members.
5. перегрузка операторов и преобразований.
6. индексаторы.

7. deconstruct.
8. target-typed new.

Практический результат: Реализовать классификацию операций без хрупкой цепочки if.

18. Практикум библиотеки предметной области

Статус: план.

1. Разделение CLI и class library.
2. агрегаты и правила.
3. интерфейсы на границах.
4. равенство.
5. коллекции только для чтения.
6. документирование API.
7. совместимость.
8. ошибки как результат и исключения.

Практический результат: Вынести ExpenseLedger.Core и подтвердить независимость от UI и диска.

III. Полная карта языка и CLR

19–24; нужны 11–18; ориентир 25–40 часов.

19. Сборки и модель исполнения

Статус: план.

1. PE, IL и metadata.
2. тип, assembly, module, namespace.
3. CTS/CLS.
4. загрузка типов и статическая инициализация.
5. JIT, tiered compilation, ReadyToRun, Native AOT.
6. public/internal и InternalsVisibleTo.
7. сборки и версии.

Практический результат: Проследить путь вызова от исходника до IL и машинного кода.

20. Память и время жизни

Статус: план.

1. Стек вызовов, managed heap и ссылки.
2. копирование значений.
3. GC roots, поколения, LOH и pinning.
4. finalizer и Dispose pattern.

5. WeakReference, SafeHandle.
6. IDisposable/IAsyncDisposable.
7. managed leaks.
8. владение ресурсом.

Практический результат: Найти удерживаемый объект и исправить время жизни ресурса.

21. Атрибуты, reflection и dynamic

Статус: план.

1. Атрибуты и targets.
2. typeof/GetType/nameof.
3. metadata inspection.
4. Activator, generics и reflection.
5. dynamic и runtime binder.
6. anonymous types.
7. expression trees.
8. trimming annotations.
9. рефлексия при AOT.
10. CallerMemberName/CallerArgumentExpression.

Практический результат: Сделать валидатор по метаданным и описать ограничения AOT.

22. Структура исходников и инструменты компилятора

Статус: план.

1. file-scoped namespaces и file-local types.
2. global/static/alias using и extern alias.
3. partial типы, методы, свойства, события, конструкторы.
4. #if/#define/#nullable/#pragma/#line/#error/#warning/#region.
5. XML docs.
6. analyzers, Roslyn syntax/semantic model, incremental source generators.

Практический результат: Добавить анализатор правила и генератор кода с тестами.

23. Ссылки, span и unsafe

Статус: план.

1. ref locals/returns, ref readonly, scoped.
2. ref struct и escape analysis.
3. Span/ReadOnlySpan, Memory/ReadOnlyMemory.
4. stackalloc, ArrayPool и ownership.

5. pointers, fixed, sizeof, function pointers, unmanaged calling conventions.
6. inline arrays.
7. MemoryMarshal и границы безопасности.

Практический результат: Сравнить выделения памяти в двух парсерах и проверить корректность.

24. Версии языка и чтение старого кода

Статус: план.

1. История возможностей C# 1–14.
2. LangVersion, TFM и SDK.
3. nullable migration.
4. records, required, collection expressions и primary constructors.
5. C# 13 params collections, Lock, ref struct interfaces.
6. C# 14 field, extension blocks, null-conditional assignment, unbound nameof, span conversions, lambda modifiers, partial members, compound operators.
7. file-based apps.
8. stable против preview.

Практический результат: Перенести старый фрагмент на современный стиль, сохранив поведение.

IV. Библиотеки .NET и внешние данные

25–30; нужны 11–18, обзор 19–20; ориентир 25–40 часов.

25. Коллекции и алгоритмы BCL

Статус: план.

1. ICollection/IList/IReadOnlyList.
2. Dictionary, SortedDictionary, SortedSet, PriorityQueue.
3. Immutable и Frozen.
4. Concurrent collections.
5. comparer и equality comparer.
6. поиск, сортировка, двоичный поиск.
7. capacity.
8. BigInteger и generic math.

Практический результат: Выбрать структуры для очереди работ и доказать порядок обработки.

26. Текст, время и регулярные выражения

Статус: план.

1. Encoding и Unicode normalization.
2. Rune/StringInfo.

3. Regex, timeout, NonBacktracking и GeneratedRegex.
4. DateTime/DateTimeOffset/DateOnly/TimeOnly/TimeSpan.
5. TimeZoneInfo и DST.
6. TimeProvider.
7. CultureInfo.
8. ресурсы локализации.
9. Guid, Random и криптографический RNG.

Практический результат: Импортировать даты с часовыми поясами и протестировать переход времени.

27. Файлы, потоки и сериализация

Статус: план.

1. File/Directory/Path, Stream, BufferedStream, async I/O.
2. JSON и System.Text.Json, converters и source generation.
3. XML и LINQ to XML.
4. CSV и CsvHelper.
5. ZIP.
6. format versions, размер входа, повреждение данных, замена файла.
7. запрет небезопасной десериализации.

Практический результат: Сделать версионируемый импорт/экспорт с проверкой испорченных данных.

28. HTTP и сетевые клиенты

Статус: план.

1. HTTP методы, статусы и заголовки.
2. HttpClient/handler lifetime.
3. IHttpClientFactory.
4. JSON, streams, timeout/cancellation.
5. retries/backoff/jitter и идемпотентность.
6. Microsoft.Extensions.Http.Resilience и Polly.
7. sockets, DNS, TLS, WebSocket обзор.

Практический результат: Написать клиент с отменой, ограниченным повтором и тестовым сервером.

29. Host, DI, настройки и журналы

Статус: план.

1. Generic Host.
2. dependency injection.
3. transient/scoped/singleton.

4. captive dependency.
5. Options и validation.
6. appsettings, environment variables и user secrets.
7. ILogger, structured logging и Serilog.
8. scope.
9. startup/shutdown.

Практический результат: Запустить импорт как службу с корректным завершением и диагностикой.

30. Практикум утилиты импорта

Статус: план.

1. Аргументы CLI и System.CommandLine.
2. Spectre.Console как необязательный UI.
3. конфигурация.
4. файлы и HTTP.
5. checkpoint.
6. обработка частично ошибочного набора.
7. отчёт об импорте.
8. повторный запуск.

Практический результат: Поставить воспроизводимый импортёр с отдельными ошибками каждой записи.

V. Асинхронность, потоки и производительность

31–35; нужны 14–15, 20 и 27–29; ориентир 30–45 часов.

31. Task и async await

Статус: план.

1. I/O-bound и CPU-bound.
2. `Task`/`Task<T>`, state machine, awaitable pattern.
3. исключения до/после await.
4. WhenAll/WhenAny.
5. CancellationTokenSource и deadline.
6. async all the way.
7. SynchronizationContext, ConfigureAwait.
8. deadlock .Result.
9. ValueTask.
10. async streams и await using.

Практический результат: Скачать набор данных с отменой и сохранением всех ошибок.

32. Потоки и синхронизация

Статус: план.

1. Thread/ThreadPool.
2. shared mutable state.
3. race и memory visibility.
4. lock и System.Threading.Lock.
5. Monitor, Interlocked, Volatile, SemaphoreSlim, Mutex.
6. deadlock и порядок захвата.
7. thread safety и reentrancy.
8. AsyncLocal.

Практический результат: Воспроизвести потерю обновления и исправить её с объяснением.

33. Параллельные конвейеры

Статус: план.

1. Parallel.ForEachAsync и PLINQ.
2. bounded concurrency.
3. Channels и backpressure.
4. producer/consumer.
5. TPL Dataflow обзор.
6. cancellation propagation.
7. completion и graceful shutdown.
8. IAsyncEnumerable.
9. периодические задачи.

Практический результат: Создать ограниченный конвейер импорта и доказать отсутствие потери записей.

34. Измерение и оптимизация

Статус: план.

1. Release и warmup.
2. BenchmarkDotNet.
3. время, allocations, throughput и percentiles.
4. dotnet-counters/trace/dump.
5. GC и threadpool starvation.
6. boxing, closure, LINQ hot path.
7. pools, spans, SIMD обзор.

8. измерение до/после.

Практический результат: Найти узкое место по профилю и подтвердить улучшение без изменения результата.

35. Практикум фонового обработчика

Статус: план.

1. BackgroundService.
2. конкурентные задания.
3. очередь.
4. лимиты.
5. отмена.
6. retries и poison message.
7. поведение при падении.
8. нагрузочный эксперимент.
9. диагностика.

Практический результат: Обработать поток операций с лимитом памяти и управляемым завершением.

VI. Инженерная работа и качество

36–39; нужны 18 и 31; ориентир 25–40 часов.

36. Автоматические тесты

Статус: план.

1. xUnit как основной пример.
2. NUnit/MSTest как альтернативы.
3. Arrange/Act/Assert.
4. parameterized tests.
5. unit/integration/contract/e2e.
6. fakes и NSubstitute.
7. FsCheck обзор.
8. coverage и mutation testing.
9. тестирование времени и async.
10. Testcontainers.

Практический результат: Проверить бизнес-правила, сбои сети и реальные ограничения БД.

37. Проектирование и изменение кода

Статус: план.

1. Cohesion/coupling, SOLID с контрпримерами.

2. composition.
3. DI boundaries.
4. layered architecture и vertical slices.
5. Strategy, Factory, Decorator, Adapter, Observer.
6. CQRS по необходимости.
7. refactoring.
8. API contracts, errors, immutability.
9. code review.

Практический результат: Изменить бизнес-правило без дублирования и большого каскада изменений.

38. Git, сборка и NuGet

Статус: план.

1. Branch, commit, diff, merge и PR.
2. csproj/MSBuild/Directory.Build.props.
3. NuGet и central package management.
4. lock file, restore.
5. transitive dependencies и аудит.
6. analyzers/.editorconfig/dotnet format.
7. SDK pinning.
8. CI.
9. pack и SemVer.

Практический результат: Собрать библиотеку и проверить её потребление из другого проекта.

39. Практикум сопровождения

Статус: план.

1. Воспроизведение бага.
2. минимальный failing test.
3. отладчик и логи.
4. совместимость.
5. миграция API.
6. README и ADR.
7. review checklist.
8. измеримое определение готовности.

Практический результат: Принять чужой код, исправить дефект и подготовить проверяемое изменение.

VII. Данные и базы данных

40–43; нужны 16, 27, 31 и 36; ориентир 25–40 часов.

40. SQL и реляционная модель

Статус: план.

1. Таблицы, ключи, constraints, NULL и трёхзначная логика.
2. SELECT/JOIN/GROUP BY.
3. индексы и query plan.
4. normalization.
5. transactions, isolation и concurrency.
6. SQLite и PostgreSQL/SQL Server.
7. migrations.

Практический результат: Спроектировать схему расходов и обосновать индексы.

41. ADO.NET и Dapper

Статус: план.

1. Connection/command/reader и pooling.
2. parameters.
3. transactions.
4. async и cancellation.
5. mapping.
6. Dapper Query/Execute.
7. SQL injection.
8. streaming vs buffering.
9. provider-specific types.

Практический результат: Сделать параметризованный репозиторий и проверить rollback.

42. Entity Framework Core

Статус: план.

1. DbContext lifetime.
2. entities и DTO.
3. mapping, navigation и relationships.
4. tracking/AsNoTracking.
5. projection.
6. eager/explicit/lazy loading и N+1.
7. LINQ translation.

8. migrations.
9. concurrency tokens.
10. transactions и bulk operations.

Практический результат: Заменить файловое хранилище БД и устранить N+1.

43. Практикум надёжного хранения

Статус: план.

1. Unique constraints.
2. optimistic concurrency.
3. keyset pagination.
4. batch import.
5. migrations в CI и развертывании.
6. Testcontainers.
7. backup/restore.
8. query logging.
9. идемпотентность.

Практический результат: Доказать отсутствие дублей и потери обновлений при конкурирующих запросах.

VIII. ASP.NET Core и сервисы

44–49; нужны 28–29, 31, 36 и 40–43; ориентир 40–65 часов.

44. HTTP API на ASP.NET Core

Статус: план.

1. Kestrel и request pipeline.
2. middleware order.
3. routing.
4. minimal APIs и controllers.
5. binding/validation.
6. DTO.
7. ProblemDetails.
8. status codes.
9. OpenAPI.
10. pagination и versioning.

Практический результат: Опубликовать локальный API с проверяемым HTTP-контрактом.

45. MVC, Razor Pages и браузер

Статус: план.

1. MVC и server rendering.
2. Razor syntax.
3. forms, model binding и validation.
4. layout/partials.
5. cookies, sessions, static assets.
6. HTML/CSS/JS минимум.
7. antiforgery и XSS.
8. различие API и браузерного приложения.

Практический результат: Создать страницу списка и форму редактирования с обработкой ошибок.

46. Идентификация и защита API

Статус: план.

1. Authentication/authorization.
2. Identity.
3. cookies, bearer, OAuth2/OIDC.
4. claims, roles, policies и ownership.
5. password hashing через стандартные средства.
6. CORS/CSRF/XSS.
7. HTTPS.
8. rate limiting.
9. секреты и журналирование.
10. threat modeling.

Практический результат: Добавить пользователей и доказать изоляцию их данных негативными тестами.

47. Кэш, фоновые задачи и наблюдаемость

Статус: план.

1. IMemoryCache/HybridCache, distributed cache и Redis.
2. invalidation и stampede.
3. BackgroundService, Quartz/Hangfire обзор.
4. health checks.
5. structured logs.
6. metrics/tracing и OpenTelemetry.
7. correlation.

8. service configuration.

Практический результат: Добавить кэш с корректной инвалидацией и найти медленный запрос по trace.

48. Связь между сервисами

Статус: план.

1. gRPC и Protobuf.
2. SignalR.
3. очереди и RabbitMQ/MassTransit обзор.
4. delivery semantics.
5. outbox/inbox.
6. duplicate handling.
7. retry storms.
8. idempotency key.
9. eventual consistency.
10. modular monolith и границы микросервисов.

Практический результат: Обработать повторно доставленное сообщение без повторного эффекта.

49. Практикум ExpenseLedger API

Статус: план.

1. API, БД, identity, импорт и фоновые задачи.
2. integration tests через WebApplicationFactory.
3. OpenAPI.
4. observability.
5. load test.
6. отказ внешнего сервиса.
7. инструкции запуска.

Практический результат: Передать сервис другому разработчику с тестами и сценарием восстановления.

IX. Прикладные специализации

50–54; выбирается одна ветка после общего ядра; ориентир 25–60 часов на ветку.

50. Windows desktop и кроссплатформенный desktop

Статус: план.

1. WinForms и WPF.
2. XAML, data binding, commands, INotifyPropertyChanged.
3. MVVM и CommunityToolkit.Mvvm.

4. dispatcher/UI thread.
5. async commands.
6. Avalonia обзор.
7. installer и updates.
8. тестируемая view model.

Практический результат: Создать desktop-клиент API с отменой запроса и состояниями загрузки.

51. Blazor

Статус: план.

1. Components, parameters, lifecycle, binding.
2. routing и forms.
3. render modes, server/WebAssembly.
4. DI scopes.
5. JS interop.
6. authentication state.
7. API client.
8. bUnit.
9. UI loading/error states.

Практический результат: Создать веб-клиент с компонентами и тестами пользовательских состояний.

52. Мобильные приложения и .NET MAUI

Статус: план.

1. Workloads и ограничения платформ.
2. XAML/MVVM.
3. navigation, permissions, lifecycle.
4. offline storage.
5. connectivity.
6. async UI.
7. packaging и signing.
8. платформенные интеграции.

Практический результат: Сделать мобильный журнал с локальной очередью синхронизации.

53. C# в игровых движках

Статус: план.

1. Unity и Godot .NET как разные среды.
2. доступные версии языка/runtime.

3. сцены, компоненты и lifecycle.
4. update loop и delta time.
5. события.
6. coroutine против Task.
7. allocations/GC на кадр.
8. serialization.
9. тестируемая игровая логика.

Практический результат: Реализовать инвентарь и сохранение без привязки правил к сцене.

54. Вычисления и обработка данных

Статус: план.

1. System.Numerics, vectors и SIMD.
2. MathNet.Numerics.
3. ML.NET и ONNX Runtime обзор.
4. tensors и batching.
5. CSV/JSON pipelines.
6. точность и воспроизводимость.
7. границы C# и Python.

Практический результат: Построить воспроизводимый анализ расходов и измерить скорость обработки.

X. Интеграция, поставка и итоговая работа

55–60; нужны общее ядро и выбранная ветка; ориентир 30–50 часов.

55. Interop и специальные среды

Статус: план.

1. P/Invoke и LibraryImport.
2. marshaling, ABI, calling convention.
3. SafeHandle и native ownership.
4. COM и Windows APIs.
5. Native AOT и trimming.
6. plugins/AssemblyLoadContext и unload.
7. process boundary.
8. sandbox misconceptions.

Практический результат: Вызвать нативную функцию и корректно освободить принадлежащий ей ресурс.

56. Существующие проекты на .NET Framework

Статус: план.

1. Framework против современного .NET.
2. packages.config, старый csproj, binding redirects и GAC.
3. AppDomain.
4. WCF, Web Forms, EF6 и ASP.NET MVC5 обзор.
5. Windows-only зависимости.
6. .NET Standard.
7. постепенная миграция и characterization tests.

Практический результат: Составить и проверить план миграции отдельной библиотеки.

57. Публикация и CI/CD

Статус: план.

1. Framework-dependent/self-contained.
2. RID.
3. single file, trimming/AOT.
4. Docker multi-stage.
5. Linux и Windows differences.
6. environment config и secrets.
7. CI test/build/publish.
8. migration strategy.
9. health/readiness.
10. rollback.

Практический результат: Развернуть приложение в чистом окружении по одной инструкции.

58. Эксплуатация и разбор сбоев

Статус: план.

1. SLO и пользовательские симптомы.
2. logs/metrics/traces.
3. CPU, memory, dump и threadpool.
4. timeout/cancellation cascade.
5. graceful shutdown.
6. backup/restore drill.
7. dependency outages.
8. incident report и исправляющий тест.

Практический результат: Воспроизвести сбой, локализовать причину и восстановить работу.

59. Итоговый проект

Статус: план.

1. Требования, архитектурное решение и ADR.
2. API или UI выбранной ветки.
3. БД.
4. импорт/интеграция.
5. auth.
6. фоновые задачи.
7. CI.
8. метрики.
9. тесты.
10. документация.
11. нагрузка.
12. review и передача.

Практический результат: Сдать ExpenseLedger v3 или равный по сложности проект по рубрике.

60. Проверка знаний и дальнейший маршрут

Статус: план.

1. Карточки по всему языку.
2. чтение незнакомого кода.
3. объяснение времени жизни и асинхронности.
4. поиск по официальной документации.
5. диагностика багов.
6. оценка компромиссов.
7. собеседование и разбор пробелов.
8. план углубления.

Практический результат: Защитить решения и самостоятельно выполнить изменение требований.

Сквозной проект и контрольные точки

Этап	Главы	Что предъявить
L01	01–10	CLI читает файл, считает отчёт и корректно отклоняет ошибочные данные
L02	11–18	Библиотека предметной области с независимыми правилами и контрактами

Этап	Главы	Что предъявить
L03	19–30	Импорт/экспорт, настройки, HTTP-клиент и управляемые ресурсы
L04	31–35	Ограниченный асинхронный конвейер с отменой и измерениями
L05	36–43	Тесты и настоящее хранилище с транзакциями и контролем конкуренции
L06	44–49	API с изоляцией пользователей, интеграционными тестами и диагностикой
L07	50–58	Клиент выбранной ветки и воспроизводимая поставка
L08	59–60	Самостоятельный проект, разбор сбоя и защита изменения требований

Каждая следующая работа сохраняет уже реализованные правила и добавляет новую ответственность. Сложность растёт от отдельной функции к работе с состоянием, затем к I/O, конкуренции и отказам внешних систем.

Карта библиотек

Это рабочий набор для курса, а не рейтинг. Сначала изучается стандартный API и задача, затем сторонний инструмент. Версию каждого внешнего пакета нужно фиксировать при написании и проверке соответствующей главы.

Задача	Основной инструмент	Сопоставление или дополнительный обзор	Главы
Данные в памяти	BCL Collections, LINQ	Immutable, Frozen	07, 15–16, 25
Форматы	System.Text.Json, XML	CsvHelper	27
HTTP и устойчивость	HttpClient, Microsoft.Extensions.Http.Resilience	Polly	28
Сборка приложения	Microsoft.Extensions.Hosting/DependencyInjection/Options/Logging	Serilog	29
CLI	System.CommandLine	Spectre.Console	30
Конкуренция	Task, Channels	TPL Dataflow	31–35
Тесты	xUnit, WebApplicationFactory	NUnit, MSTest, NSubstitute, FsCheck, Testcontainers	36, 43, 49
Измерения	BenchmarkDotNet, dotnet diagnostics	OpenTelemetry	34, 47, 58
Доступ к БД	EF Core, ADO.NET	Dapper	40–43
Веб	ASP.NET Core, Identity, OpenAPI	FluentValidation как вариант валидации	44–49
Интеграция	gRPC, SignalR	Redis, RabbitMQ, MassTransit, Quartz, Hangfire	47–48
UI	WPF/WinForms, Blazor, MAUI	Avalonia, CommunityToolkit.Mvvm, bUnit	50–52

Задача	Основной инструмент	Сопоставление или дополнительный обзор	Главы
Игры	Unity или Godot .NET	API конкретного движка	53
Вычисления	System.Numerics	MathNet.Numerics, ML.NET, ONNX Runtime	54

Критерии завершения всего курса

- Учащийся объясняет семантику значений и ссылок, null, обобщений, LINQ, исключений, async и освобождения ресурсов.
- Пишет приложение с внешними данными и выбранным UI/API, самостоятельно находит нужные API в документации.
- Проверяет корректность, границы, отмену, конкуренцию и отказ зависимостей; умеет читать чужие тесты.
- Пользуется Git, отладчиком, профилировщиком, пакетами и CI; воспроизводит сборку на чистой машине.
- Обосновывает модель БД, границы ответственности и решения по безопасности и производительности.
- Передаёт работоспособный проект с инструкцией, тестами, диагностикой и описанием ограничений.

Полнота языка контролируется картой покрытия. Работа во всех существующих C#-средах требует знания их собственных API; курс даёт общее ядро и маршруты специализации.